

EE 446 TinyML - Lab 8 Part II: Tiny Ensemble Learning

This notebook demonstrates a complete TinyML ensemble-learning pipeline for IMU-based human activity classification. The code is already implemented. Your task is to run the notebook, understand the pipeline, and answer the discussion questions at the end.

The pipeline uses the `mHealth_subject6.log` dataset and builds three model branches using different input representations:

- Raw IMU windows
- Standard-scaled IMU windows
- Min-max-scaled IMU windows

Each branch trains an autoencoder, uses the encoder output as a latent feature vector, and trains a classifier. The three classifier softmax outputs are then concatenated and passed into a small stacked meta-classifier. The notebook then compresses selected models using pruning, quantization-aware training (QAT), and full integer TFLite conversion for TinyML deployment.

This notebook is designed for the local `tinymL-arduino` environment.

Recommended way to start JupyterLab:

```
source ~/ai/projects/tinymL-arduino/bin/activate
jupyter lab
```

Required files in the same folder as this notebook:

- `mHealth_subject6.log`
- Optional for the final validation section: `imu_500_rows.csv`

You do not need to fill in missing code. Run the cells in order and use the results to answer the discussion questions.

Discussion Questions

Submit your answers for Part II in the same PDF used for Lab 8 Part I.

Question 1: Model architecture and ensemble flow

Draw a complete diagram of the full ensemble pipeline. Your diagram should include enough detail for someone to reconstruct the model architecture from your drawing.

Include the following details:

- The input window size and number of input neurons. In this notebook, each input window has 100 time steps and 6 IMU features, so the flattened input dimension is 600.

- The three input branches: raw input, standard-scaled input, and min-max-scaled input.
- The encoder architecture in each branch, including the number of hidden layers, number of neurons in each layer, and activation function used in each layer.
- The classifier architecture in each branch, including the number of input neurons, hidden-layer neurons, output neurons, and activation functions.
- The number of models used in the ensemble and how they are connected.
- The size of each branch output and how the three outputs are concatenated before being passed into the stacked meta-classifier.
- The stacked meta-classifier architecture, including input size, hidden layer size, output size, and activation functions.

Question 2: Pruning and QAT methodology

Explain how this notebook compresses the models for TinyML deployment.

In your answer, describe:

- How pruning is applied and what sparsity schedule is used.
- Why the pruning wrappers are stripped before QAT.
- How QAT is applied after pruning.
- Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.
- How the final model is converted to an int8 TFLite model and why representative data is needed.
- How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.

Question 3: Arduino deployment and real-world behavior

Deploy the quantized model using the provided Arduino sketch for this lab. Place the Arduino Nano 33 BLE Sense flat on a stable surface, open the serial monitor, and observe the predicted activity labels.

In your answer, include:

- A screenshot of the serial monitor output.
- A short explanation of whether the output is acceptable for the resting board condition.
- Any issues you observe, such as unstable predictions, unexpected labels, sensor noise, scaling mismatch, or mismatch between the training data and the real board orientation.
- One or two practical changes that could improve deployment reliability.

Question 4: Deployment Behavior on the Arduino

After deploying the ensemble model on the Arduino Nano 33 BLE Sense / Sense Rev2, did you observe any unexpected prediction behavior? For example, did the model repeatedly predict the same class even when the board was moved differently?

Briefly explain why this may happen. In your answer, discuss possible causes such as differences between the original training dataset and the live Arduino sensor data, sensor placement, axis

orientation, unit conversion, feature scaling, or mismatch between the motion patterns used during training and testing.

1. The ensemble model flattens a 100×6 IMU data window into a 600-dimensional vector and processes it in parallel across three branches: raw, standard-scaled, and min-max scaled. Each branch uses a ReLU-activated encoder ($600 \rightarrow 64 \rightarrow 32$) and a softmax classifier ($32 \rightarrow 20 \rightarrow 12$) to produce a 12-class probability vector. These three outputs are concatenated into a 36-dimensional vector and fed into a final stacked meta-classifier ($36 \rightarrow 24 \rightarrow 12$) to generate the ultimate prediction. By combining three encoders, three classifiers, and one meta-classifier, this seven-model system leverages diverse data distributions to enhance overall accuracy and robustness.
2. To optimize the model for resource-constrained TinyML microcontrollers, it undergoes a compression pipeline that begins with magnitude-based pruning, gradually increasing weight sparsity from 10% to 80% over 500 training steps. After stripping the pruning wrappers to eliminate structural overhead, the model undergoes Quantization Aware Training (QAT) to adapt to low-precision arithmetic, utilizing a custom callback to ensure pruned weights stay exactly at zero. Finally, a representative dataset is used to calibrate real activation ranges, allowing the model to be converted into a fully int8 TensorFlow Lite format. This combination of pruning and quantization drastically reduces the model's memory and computational requirements without sacrificing strong classification performance.
3. The TinyML ensemble pipeline successfully executes real time inference on the Arduino, processing 100 sample IMU windows per cycle within the device hardware constraints. For the resting board condition, the overall output is acceptable. Despite internal disagreements among the submodels, the stacked meta classifier correctly and consistently generates a final prediction of "Standing still" (class 0), proving that the complete preprocessing and classification pipeline functions properly on the embedded device. However, there are noticeable issues regarding model confidence and conflicting branch outputs. The raw data classifier produces an unexpected, highly confident prediction for class 11 (approximately 0.98 confidence), which heavily conflicts with the standard scaled and min max scaled branches. This discrepancy strongly suggests a baseline offset, such as sensor noise or an orientation mismatch between the live board on your desk and the original training data. Because the meta classifier must synthesize these conflicting inputs, the final ensemble confidence drops to a relatively low 0.39. While the prediction remained stable during testing, this narrow confidence margin leaves the system vulnerable to erratic jumps if the sensor noise changes slightly. To improve deployment reliability, a couple of practical changes could be implemented. First, introducing temporal smoothing, such as a majority vote filter across three to five consecutive inference windows, would stabilize the output and prevent flickering between states. Second, adding a startup calibration routine when the board boots up to capture resting sensor values and calculate an offset would help align the live physical data with the feature distributions expected by the trained models, thereby reducing branch conflict and boosting final confidence.

4. Yes, During deployment, the ensemble model exhibited unexpected behavior by repeatedly predicting "Standing still" across multiple inference windows, regardless of how the board was actually moved. This issue primarily stems from a domain shift between the original mHealth training dataset and the live data collected by the Arduino Nano 33 BLE Sense. Several factors contribute to this discrepancy, including differences in sensor hardware such as accelerometer sensitivity and noise profiles, as well as mismatches in board orientation that alter the gravity vectors on the X, Y, and Z axes. Furthermore, slight deviations in real time preprocessing, standard scaling, or unit conversions can cause the live inputs to fall outside the ranges the model learned during training. Combined with the fact that live testing motions rarely perfectly replicate the exact movements of the original dataset participants, the model struggles to interpret the unfamiliar incoming data. Consequently, the ensemble defaults to a "safe" or familiar baseline class, resulting in reduced confidence, distinct class bias, and repetitive predictions.

1. Environment check

This cell verifies that the notebook is running inside the intended Python environment and that the required libraries are available. It does not install or uninstall packages.

```
import os
os.environ.setdefault("KERAS_BACKEND", "tensorflow")

import sys
import platform
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

try:
    import tensorflow_model_optimization as tfmot
except ImportError as exc:
    raise ImportError(
        "tensorflow_model_optimization is not installed in this environment. "
        "Activate tinymml-arduino and install tensorflow-model-optimization before running this notebook."
    ) from exc

print("Python executable:", sys.executable)
print("Python version:", platform.python_version())
print("TensorFlow version:", tf.__version__)
print("TF-MOT version:", tfmot.__version__)
```

```
2026-05-28 20:29:01.898471: I tensorflow/core/util/port.cc:111] oneDNN custom operations are on. You may see slightly different numerical
```

```

results due to floating-point round-off errors from different
computation orders. To turn them off, set the environment variable
`TF_ENABLE_ONEDNN_OPTS=0`.
2026-05-28 20:29:01.982780: I tensorflow/tsl/cuda/cudart_stub.cc:28]
Could not find cuda drivers on your machine, GPU will not be used.
2026-05-28 20:29:02.347762: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable
to register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
2026-05-28 20:29:02.347816: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable
to register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
2026-05-28 20:29:02.350835: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable
to register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
2026-05-28 20:29:02.564372: I tensorflow/tsl/cuda/cudart_stub.cc:28]
Could not find cuda drivers on your machine, GPU will not be used.
2026-05-28 20:29:02.567985: I
tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow
binary is optimized to use available CPU instructions in performance-
critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other
operations, rebuild TensorFlow with the appropriate compiler flags.
2026-05-28 20:29:04.285174: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning:
Could not find TensorRT

Python executable: /home/metal/ai/projects/tinymml-arduino/bin/python
Python version: 3.11.15
TensorFlow version: 2.14.1
TF-MOT version: 0.8.0

```

2. Load and prepare the mHealth dataset

The mHealth subject 6 log file contains multiple sensor channels and an activity label. This lab uses six IMU features: three accelerometer channels and three gyroscope channels. The label column is converted from 1-based labels to 0-based labels for TensorFlow training.

```

DATA_FILE = Path("mHealth_subject6.log")

if not DATA_FILE.exists():
    raise FileNotFoundError(
        "mHealth_subject6.log was not found. Place the file in the
        same folder as this notebook and rerun this cell."
    )

sensor_features = ["Accel_X", "Accel_Y", "Accel_Z", "Gyro_X",

```

```

"Gyro_Y", "Gyro_Z"]
sensor_cols = {
    5: "Accel_X", 6: "Accel_Y", 7: "Accel_Z",
    8: "Gyro_X", 9: "Gyro_Y", 10: "Gyro_Z"
}

label_map = {
    0: "Standing still",
    1: "Sitting and relaxing",
    2: "Lying down",
    3: "Walking",
    4: "Climbing stairs",
    5: "Waist bends forward",
    6: "Frontal elevation of arms",
    7: "Knees bending (crouching)",
    8: "Cycling",
    9: "Jogging",
    10: "Running",
    11: "Jump front and back",
}

raw_df = pd.read_csv(DATA_FILE, sep="\t", header=None)
raw_df = raw_df[raw_df[23] > 0].copy()
raw_df.rename(columns=sensor_cols, inplace=True)

sensor_df = raw_df[sensor_features].copy()
sensor_df["Label"] = raw_df[23].to_numpy(dtype=int) - 1

print("Dataset shape:", sensor_df.shape)
print("Class labels:", sorted(sensor_df["Label"].unique()))
print("\nOriginal feature statistics:")
display(sensor_df[sensor_features].describe())

```

Dataset shape: (32205, 7)

Class labels: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

Original feature statistics:

	Accel_X	Accel_Y	Accel_Z	Gyro_X	Gyro_Y	Gyro_Z
count	32205.000000	32205.000000	32205.000000	32205.000000	32205.000000	32205.000000
mean	3.268805	-9.153526	0.700608	-0.020571	-0.520382	-0.571107
std	4.253270	5.397648	6.433731	0.325434	1.611600	1.611600
min	-22.091000	-19.571000	-19.363000	-0.977740	-1.611600	-1.611600
25%	1.284000	-10.420000	-1.359900	-0.235620	-0.863040	-0.863040

50%	2.427500	-9.583100	1.509900	-0.027829	-
0.731710					
75%	4.725200	-8.039700	2.456600	0.192950	-
0.452160					
max	20.003000	20.909000	24.599000	1.463800	
1.557200					

	Gyro_Z
count	32205.000000
mean	-0.299817
std	0.542112
min	-1.206300
25%	-0.662080
50%	-0.479370
75%	0.341850
max	1.326100

3. Create raw, standard-scaled, and min-max-scaled datasets

The ensemble uses three versions of the same IMU signal. This creates diversity across the branches without changing the underlying activity labels.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

X_features = sensor_df[sensor_features].to_numpy(dtype=np.float32)
y_labels = sensor_df["Label"].to_numpy(dtype=int)

std_scaler = StandardScaler()
X_std = std_scaler.fit_transform(X_features).astype(np.float32)

minmax_scaler = MinMaxScaler()
X_minmax = minmax_scaler.fit_transform(X_features).astype(np.float32)

sensor_df_raw = pd.DataFrame(X_features, columns=sensor_features)
sensor_df_raw["Label"] = y_labels

sensor_df_std = pd.DataFrame(X_std, columns=sensor_features)
sensor_df_std["Label"] = y_labels

sensor_df_minmax = pd.DataFrame(X_minmax, columns=sensor_features)
sensor_df_minmax["Label"] = y_labels

print("Standard-scaled feature statistics:")
display(sensor_df_std[sensor_features].describe())

print("Min-max-scaled feature statistics:")
display(sensor_df_minmax[sensor_features].describe())
```

Standard-scaled feature statistics:

	Accel_X	Accel_Y	Accel_Z	Gyro_X
Gyro_Y \				
count	32205.000000	3.220500e+04	32205.000000	3.220500e+04
mean	0.000000	-1.061316e-07	0.000000	-1.516166e-08
std	1.000015	1.000015e+00	1.000015	1.000015e+00
min	-5.962517	-1.930032e+00	-3.118551	-2.941250e+00
25%	-0.466661	-2.346382e-01	-0.320271	-6.608158e-01
50%	-0.197805	-7.958682e-02	0.125791	-2.230210e-02
75%	0.342423	2.063571e-01	0.272940	6.561219e-01
max	3.934491	5.569647e+00	3.714605	4.561272e+00

	Gyro_Z
count	3.220500e+04
mean	7.580830e-09
std	1.000015e+00
min	-1.672159e+00
25%	-6.682549e-01
50%	-3.312160e-01
75%	1.183661e+00
max	2.999273e+00

Min-max-scaled feature statistics:

	Accel_X	Accel_Y	Accel_Z	Gyro_X
Gyro_Y \				
count	32205.000000	32205.000000	32205.000000	32205.000000
mean	0.602457	0.257349	0.456385	0.392035
std	0.101042	0.133341	0.146348	0.133291
min	0.000000	0.000000	0.000000	0.000000
25%	0.555305	0.226062	0.409515	0.303956
50%	0.582470	0.246737	0.474794	0.389062
75%	0.637055	0.284864	0.496329	0.479488
max	1.000000	1.000000	1.000000	1.000000

1.000000

	Gyro_Z
count	32205.000000
mean	0.357954
std	0.214070
min	0.000000
25%	0.214903
50%	0.287052
75%	0.611337
max	1.000000

4. Convert the time series into flattened windows

Each sample is a 100-time-step window with 6 sensor features. The model input dimension is therefore:

100 time steps x 6 features = 600 input neurons

The split is done within each class to preserve the temporal structure of each activity segment.

```
WINDOW_SIZE = 100
STRIDE = 1
NUM_CLASSES = 12

def create_windows(X_seq, label, window_size=WINDOW_SIZE,
                  stride=STRIDE):
    windows = []
    labels = []
    for start in range(0, len(X_seq) - window_size + 1, stride):
        window = X_seq[start:start + window_size].reshape(-1)
        windows.append(window)
        labels.append(label)
    return windows, labels

def window_and_split(df_scaled, train_fraction=0.7):
    X_train_windows, X_test_windows = [], []
    y_train_labels, y_test_labels = [], []

    for label in sorted(df_scaled["Label"].unique()):
        df_class = df_scaled[df_scaled["Label"] == label]
        X_class = df_class[sensor_features].to_numpy(dtype=np.float32)
        split_idx = int(train_fraction * len(X_class))

        train_windows, train_labels =
        create_windows(X_class[:split_idx], label)
        test_windows, test_labels =
        create_windows(X_class[split_idx:], label)
```

```

X_train_windows.extend(train_windows)
y_train_labels.extend(train_labels)
X_test_windows.extend(test_windows)
y_test_labels.extend(test_labels)

return (
    np.asarray(X_train_windows, dtype=np.float32),
    np.asarray(X_test_windows, dtype=np.float32),
    np.asarray(y_train_labels, dtype=int),
    np.asarray(y_test_labels, dtype=int),
)

X_train_raw, X_test_raw, y_train_raw, y_test_raw =
window_and_split(sensor_df_raw)
X_train_std, X_test_std, y_train_std, y_test_std =
window_and_split(sensor_df_std)
X_train_minmax, X_test_minmax, y_train_minmax, y_test_minmax =
window_and_split(sensor_df_minmax)

print("Raw input:", X_train_raw.shape, X_test_raw.shape)
print("Standard-scaled input:", X_train_std.shape, X_test_std.shape)
print("Min-max-scaled input:", X_train_minmax.shape,
X_test_minmax.shape)

Raw input: (21350, 600) (8479, 600)
Standard-scaled input: (21350, 600) (8479, 600)
Min-max-scaled input: (21350, 600) (8479, 600)

```

5. Visualize the three input representations with t-SNE

This visualization helps compare how the raw, standard-scaled, and min-max-scaled windows are distributed before training the autoencoders. To keep the notebook practical on a local machine, the plot uses a stratified subset of test windows.

```

from sklearn.manifold import TSNE

TSNE_MAX_SAMPLES = 2000

def stratified_subset(X, y, max_samples=TSNE_MAX_SAMPLES,
random_state=42):
    rng = np.random.default_rng(random_state)
    labels = np.unique(y)
    per_class = max(1, max_samples // len(labels))
    chosen = []

    for label in labels:
        idx = np.where(y == label)[0]

```

```

        sample_size = min(per_class, len(idx))
        chosen.extend(rng.choice(idx, size=sample_size,
replace=False))

    chosen = np.asarray(chosen)
    rng.shuffle(chosen)
    return X[chosen], y[chosen]

def compute_tsne(X, random_state=42):
    kwargs = dict(n_components=2, perplexity=30, learning_rate=300,
random_state=random_state)
    try:
        return TSNE(max_iter=1000, **kwargs).fit_transform(X)
    except TypeError:
        return TSNE(n_iter=1000, **kwargs).fit_transform(X)

def run_tsne_and_plot(X_test, y_test, title, ax):
    X_sub, y_sub = stratified_subset(X_test, y_test)
    X_tsne = compute_tsne(X_sub)

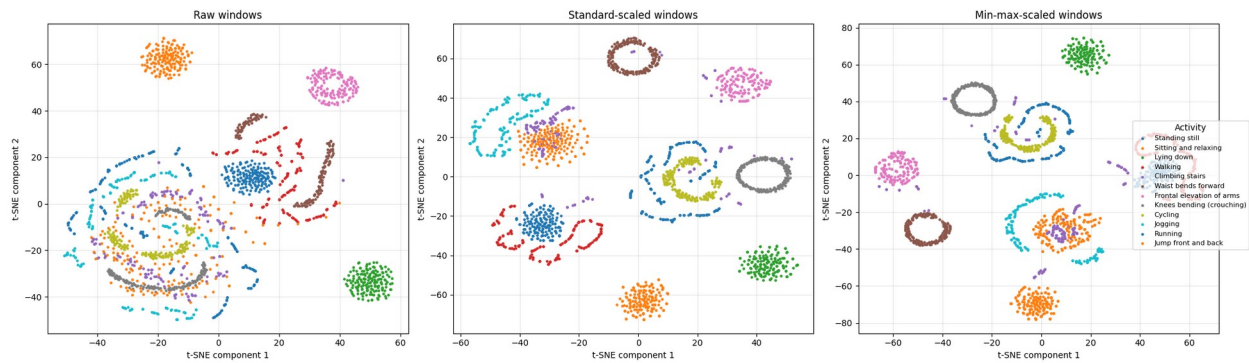
    for label in sorted(np.unique(y_sub)):
        idx = y_sub == label
        ax.scatter(X_tsne[idx, 0], X_tsne[idx, 1], s=5,
label=label_map[label])

    ax.set_title(title)
    ax.set_xlabel("t-SNE component 1")
    ax.set_ylabel("t-SNE component 2")
    ax.grid(True, alpha=0.3)

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
run_tsne_and_plot(X_test_raw, y_test_raw, "Raw windows", axes[0])
run_tsne_and_plot(X_test_std, y_test_std, "Standard-scaled windows",
axes[1])
run_tsne_and_plot(X_test_minmax, y_test_minmax, "Min-max-scaled
windows", axes[2])

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, title="Activity", loc="center right",
bbox_to_anchor=(1.02, 0.5), fontsize=8)
plt.tight_layout()
plt.show()

```



6. Train one autoencoder for each input representation

Each autoencoder reconstructs its own input representation. The encoder part is then used as a compact feature extractor.

Encoder architecture for each branch:

600 input neurons -> Dense(64, ReLU) -> Dense(32, linear)

Decoder architecture for each branch:

32 latent neurons -> Dense(64, ReLU) -> Dense(600, linear)

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import Adam

AUTOENCODER_EPOCHS = 50
BATCH_SIZE = 256
LATENT_DIM = 32

def build_autoencoder(input_dim, latent_dim=LATENT_DIM):
    input_layer = Input(shape=(input_dim,), name="input_window")
    encoded = Dense(64, activation="relu", name="encoder_dense_64")(input_layer)
    latent = Dense(latent_dim, activation="linear", name="latent_32")(encoded)
    decoded = Dense(64, activation="relu", name="decoder_dense_64")(latent)
    reconstructed = Dense(input_dim, activation="linear", name="reconstruction")(decoded)

    autoencoder = Model(inputs=input_layer, outputs=reconstructed, name="autoencoder")
    encoder = Model(inputs=input_layer, outputs=latent, name="encoder")
    autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss="mse")
```

```

    return autoencoder, encoder

def train_autoencoder_branch(X_train, branch_name):
    input_dim = X_train.shape[1]
    autoencoder, encoder = build_autoencoder(input_dim)
    print(f"Training autoencoder for {branch_name} input")
    autoencoder.fit(
        X_train,
        X_train,
        epochs=AUTOENCODER_EPOCHS,
        batch_size=BATCH_SIZE,
        shuffle=True,
        verbose=1,
    )
    return autoencoder, encoder

models = {
    "raw": train_autoencoder_branch(X_train_raw, "raw"),
    "std": train_autoencoder_branch(X_train_std, "standard-scaled"),
    "minmax": train_autoencoder_branch(X_train_minmax, "min-max-
scaled"),
}

```

```

2026-05-28 20:29:22.748533: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:880]
could not open file to read NUMA node:
/sys/bus/pci/devices/0000:01:00.0/numa_node
Your kernel may have been built without NUMA support.
2026-05-28 20:29:22.749118: W
tensorflow/core/common_runtime/gpu/gpu_device.cc:2211] Cannot dlopen
some GPU libraries. Please make sure the missing libraries mentioned
above are installed properly if you would like to use GPU. Follow the
guide at https://www.tensorflow.org/install/gpu for how to download
and setup the required libraries for your platform.
Skipping registering GPU devices...

```

```

Training autoencoder for raw input
Epoch 1/50
84/84 [=====] - 1s 7ms/step - loss: 14.6551
Epoch 2/50
84/84 [=====] - 0s 5ms/step - loss: 8.4095
Epoch 3/50
84/84 [=====] - 0s 5ms/step - loss: 7.5707
Epoch 4/50
84/84 [=====] - 0s 3ms/step - loss: 6.9405
Epoch 5/50
84/84 [=====] - 0s 4ms/step - loss: 6.4752
Epoch 6/50
84/84 [=====] - 1s 6ms/step - loss: 6.1536

```

```
Epoch 7/50
84/84 [=====] - 0s 6ms/step - loss: 5.9054
Epoch 8/50
84/84 [=====] - 1s 7ms/step - loss: 5.6996
Epoch 9/50
84/84 [=====] - 1s 7ms/step - loss: 5.5288
Epoch 10/50
84/84 [=====] - 0s 5ms/step - loss: 5.3898
Epoch 11/50
84/84 [=====] - 0s 3ms/step - loss: 5.2754
Epoch 12/50
84/84 [=====] - 0s 3ms/step - loss: 5.1815
Epoch 13/50
84/84 [=====] - 1s 7ms/step - loss: 5.1065
Epoch 14/50
84/84 [=====] - 1s 7ms/step - loss: 5.0375
Epoch 15/50
84/84 [=====] - 1s 8ms/step - loss: 4.9777
Epoch 16/50
84/84 [=====] - 1s 7ms/step - loss: 4.9230
Epoch 17/50
84/84 [=====] - 0s 4ms/step - loss: 4.8760
Epoch 18/50
84/84 [=====] - 0s 5ms/step - loss: 4.8375
Epoch 19/50
84/84 [=====] - 0s 6ms/step - loss: 4.7989
Epoch 20/50
84/84 [=====] - 0s 5ms/step - loss: 4.7628
Epoch 21/50
84/84 [=====] - 0s 6ms/step - loss: 4.7291
Epoch 22/50
84/84 [=====] - 0s 6ms/step - loss: 4.7013
Epoch 23/50
84/84 [=====] - 0s 5ms/step - loss: 4.6745
Epoch 24/50
84/84 [=====] - 0s 4ms/step - loss: 4.6510
Epoch 25/50
84/84 [=====] - 0s 6ms/step - loss: 4.6276
Epoch 26/50
84/84 [=====] - 0s 6ms/step - loss: 4.6075
Epoch 27/50
84/84 [=====] - 1s 6ms/step - loss: 4.5898
Epoch 28/50
84/84 [=====] - 1s 7ms/step - loss: 4.5749
Epoch 29/50
84/84 [=====] - 1s 6ms/step - loss: 4.5557
Epoch 30/50
84/84 [=====] - 0s 4ms/step - loss: 4.5408
Epoch 31/50
```

```
84/84 [=====] - 1s 7ms/step - loss: 4.5266
Epoch 32/50
84/84 [=====] - 1s 6ms/step - loss: 4.5129
Epoch 33/50
84/84 [=====] - 1s 7ms/step - loss: 4.5038
Epoch 34/50
84/84 [=====] - 1s 6ms/step - loss: 4.4925
Epoch 35/50
84/84 [=====] - 0s 5ms/step - loss: 4.4826
Epoch 36/50
84/84 [=====] - 0s 3ms/step - loss: 4.4698
Epoch 37/50
84/84 [=====] - 0s 3ms/step - loss: 4.4607
Epoch 38/50
84/84 [=====] - 1s 6ms/step - loss: 4.4501
Epoch 39/50
84/84 [=====] - 1s 6ms/step - loss: 4.4433
Epoch 40/50
84/84 [=====] - 1s 6ms/step - loss: 4.4322
Epoch 41/50
84/84 [=====] - 0s 6ms/step - loss: 4.4249
Epoch 42/50
84/84 [=====] - 0s 5ms/step - loss: 4.4161
Epoch 43/50
84/84 [=====] - 0s 3ms/step - loss: 4.4100
Epoch 44/50
84/84 [=====] - 0s 4ms/step - loss: 4.4005
Epoch 45/50
84/84 [=====] - 1s 8ms/step - loss: 4.3935
Epoch 46/50
84/84 [=====] - 1s 6ms/step - loss: 4.3861
Epoch 47/50
84/84 [=====] - 1s 7ms/step - loss: 4.3803
Epoch 48/50
84/84 [=====] - 3s 42ms/step - loss: 4.3731
Epoch 49/50
84/84 [=====] - 0s 5ms/step - loss: 4.3665
Epoch 50/50
84/84 [=====] - 0s 3ms/step - loss: 4.3581
Training autoencoder for standard-scaled input
Epoch 1/50
84/84 [=====] - 1s 6ms/step - loss: 0.5659
Epoch 2/50
84/84 [=====] - 1s 7ms/step - loss: 0.3372
Epoch 3/50
84/84 [=====] - 1s 7ms/step - loss: 0.2906
Epoch 4/50
84/84 [=====] - 0s 4ms/step - loss: 0.2629
Epoch 5/50
```

```
84/84 [=====] - 0s 3ms/step - loss: 0.2463
Epoch 6/50
84/84 [=====] - 0s 5ms/step - loss: 0.2348
Epoch 7/50
84/84 [=====] - 0s 6ms/step - loss: 0.2262
Epoch 8/50
84/84 [=====] - 1s 7ms/step - loss: 0.2197
Epoch 9/50
84/84 [=====] - 1s 7ms/step - loss: 0.2146
Epoch 10/50
84/84 [=====] - 1s 6ms/step - loss: 0.2104
Epoch 11/50
84/84 [=====] - 0s 5ms/step - loss: 0.2070
Epoch 12/50
84/84 [=====] - 0s 3ms/step - loss: 0.2042
Epoch 13/50
84/84 [=====] - 0s 3ms/step - loss: 0.2016
Epoch 14/50
84/84 [=====] - 0s 4ms/step - loss: 0.1995
Epoch 15/50
84/84 [=====] - 1s 8ms/step - loss: 0.1975
Epoch 16/50
84/84 [=====] - 1s 6ms/step - loss: 0.1959
Epoch 17/50
84/84 [=====] - 0s 6ms/step - loss: 0.1944
Epoch 18/50
84/84 [=====] - 0s 4ms/step - loss: 0.1930
Epoch 19/50
84/84 [=====] - 0s 4ms/step - loss: 0.1917
Epoch 20/50
84/84 [=====] - 0s 3ms/step - loss: 0.1906
Epoch 21/50
84/84 [=====] - 0s 3ms/step - loss: 0.1895
Epoch 22/50
84/84 [=====] - 0s 4ms/step - loss: 0.1886
Epoch 23/50
84/84 [=====] - 0s 6ms/step - loss: 0.1878
Epoch 24/50
84/84 [=====] - 1s 6ms/step - loss: 0.1871
Epoch 25/50
84/84 [=====] - 0s 4ms/step - loss: 0.1862
Epoch 26/50
84/84 [=====] - 0s 3ms/step - loss: 0.1856
Epoch 27/50
84/84 [=====] - 0s 5ms/step - loss: 0.1849
Epoch 28/50
84/84 [=====] - 0s 5ms/step - loss: 0.1843
Epoch 29/50
84/84 [=====] - 0s 5ms/step - loss: 0.1839
```



```
Epoch 30/50
84/84 [=====] - 0s 5ms/step - loss: 0.1834
Epoch 31/50
84/84 [=====] - 0s 5ms/step - loss: 0.1829
Epoch 32/50
84/84 [=====] - 0s 4ms/step - loss: 0.1824
Epoch 33/50
84/84 [=====] - 0s 3ms/step - loss: 0.1820
Epoch 34/50
84/84 [=====] - 0s 4ms/step - loss: 0.1816
Epoch 35/50
84/84 [=====] - 0s 5ms/step - loss: 0.1812
Epoch 36/50
84/84 [=====] - 0s 5ms/step - loss: 0.1810
Epoch 37/50
84/84 [=====] - 0s 5ms/step - loss: 0.1806
Epoch 38/50
84/84 [=====] - 0s 4ms/step - loss: 0.1802
Epoch 39/50
84/84 [=====] - 0s 3ms/step - loss: 0.1800
Epoch 40/50
84/84 [=====] - 0s 3ms/step - loss: 0.1797
Epoch 41/50
84/84 [=====] - 0s 3ms/step - loss: 0.1794
Epoch 42/50
84/84 [=====] - 0s 4ms/step - loss: 0.1791
Epoch 43/50
84/84 [=====] - 0s 4ms/step - loss: 0.1790
Epoch 44/50
84/84 [=====] - 0s 6ms/step - loss: 0.1787
Epoch 45/50
84/84 [=====] - 0s 5ms/step - loss: 0.1785
Epoch 46/50
84/84 [=====] - 0s 5ms/step - loss: 0.1783
Epoch 47/50
84/84 [=====] - 0s 6ms/step - loss: 0.1782
Epoch 48/50
84/84 [=====] - 0s 3ms/step - loss: 0.1780
Epoch 49/50
84/84 [=====] - 0s 4ms/step - loss: 0.1777
Epoch 50/50
84/84 [=====] - 0s 6ms/step - loss: 0.1776
Training autoencoder for min-max-scaled input
Epoch 1/50
84/84 [=====] - 1s 4ms/step - loss: 0.0427
Epoch 2/50
84/84 [=====] - 0s 5ms/step - loss: 0.0099
Epoch 3/50
84/84 [=====] - 0s 3ms/step - loss: 0.0084
```

```
Epoch 4/50
84/84 [=====] - 0s 4ms/step - loss: 0.0077
Epoch 5/50
84/84 [=====] - 0s 5ms/step - loss: 0.0069
Epoch 6/50
84/84 [=====] - 0s 4ms/step - loss: 0.0065
Epoch 7/50
84/84 [=====] - 0s 5ms/step - loss: 0.0062
Epoch 8/50
84/84 [=====] - 0s 5ms/step - loss: 0.0059
Epoch 9/50
84/84 [=====] - 0s 4ms/step - loss: 0.0057
Epoch 10/50
84/84 [=====] - 0s 4ms/step - loss: 0.0055
Epoch 11/50
84/84 [=====] - 0s 3ms/step - loss: 0.0053
Epoch 12/50
84/84 [=====] - 0s 4ms/step - loss: 0.0052
Epoch 13/50
84/84 [=====] - 1s 6ms/step - loss: 0.0050
Epoch 14/50
84/84 [=====] - 1s 6ms/step - loss: 0.0049
Epoch 15/50
84/84 [=====] - 1s 6ms/step - loss: 0.0048
Epoch 16/50
84/84 [=====] - 1s 7ms/step - loss: 0.0047
Epoch 17/50
84/84 [=====] - 0s 5ms/step - loss: 0.0046
Epoch 18/50
84/84 [=====] - 0s 3ms/step - loss: 0.0045
Epoch 19/50
84/84 [=====] - 0s 4ms/step - loss: 0.0044
Epoch 20/50
84/84 [=====] - 0s 5ms/step - loss: 0.0044
Epoch 21/50
84/84 [=====] - 0s 5ms/step - loss: 0.0043
Epoch 22/50
84/84 [=====] - 0s 4ms/step - loss: 0.0043
Epoch 23/50
84/84 [=====] - 0s 4ms/step - loss: 0.0042
Epoch 24/50
84/84 [=====] - 0s 4ms/step - loss: 0.0042
Epoch 25/50
84/84 [=====] - 0s 4ms/step - loss: 0.0041
Epoch 26/50
84/84 [=====] - 3s 38ms/step - loss: 0.0041
Epoch 27/50
84/84 [=====] - 0s 3ms/step - loss: 0.0041
Epoch 28/50
```

```
84/84 [=====] - 0s 5ms/step - loss: 0.0040
Epoch 29/50
84/84 [=====] - 0s 5ms/step - loss: 0.0040
Epoch 30/50
84/84 [=====] - 0s 5ms/step - loss: 0.0040
Epoch 31/50
84/84 [=====] - 0s 5ms/step - loss: 0.0039
Epoch 32/50
84/84 [=====] - 0s 4ms/step - loss: 0.0039
Epoch 33/50
84/84 [=====] - 0s 4ms/step - loss: 0.0039
Epoch 34/50
84/84 [=====] - 0s 3ms/step - loss: 0.0039
Epoch 35/50
84/84 [=====] - 0s 4ms/step - loss: 0.0038
Epoch 36/50
84/84 [=====] - 0s 6ms/step - loss: 0.0038
Epoch 37/50
84/84 [=====] - 0s 5ms/step - loss: 0.0038
Epoch 38/50
84/84 [=====] - 1s 6ms/step - loss: 0.0038
Epoch 39/50
84/84 [=====] - 0s 6ms/step - loss: 0.0038
Epoch 40/50
84/84 [=====] - 0s 5ms/step - loss: 0.0038
Epoch 41/50
84/84 [=====] - 0s 3ms/step - loss: 0.0037
Epoch 42/50
84/84 [=====] - 0s 6ms/step - loss: 0.0037
Epoch 43/50
84/84 [=====] - 0s 5ms/step - loss: 0.0037
Epoch 44/50
84/84 [=====] - 0s 6ms/step - loss: 0.0037
Epoch 45/50
84/84 [=====] - 0s 6ms/step - loss: 0.0037
Epoch 46/50
84/84 [=====] - 0s 5ms/step - loss: 0.0037
Epoch 47/50
84/84 [=====] - 0s 3ms/step - loss: 0.0037
Epoch 48/50
84/84 [=====] - 0s 4ms/step - loss: 0.0036
Epoch 49/50
84/84 [=====] - 0s 5ms/step - loss: 0.0036
Epoch 50/50
84/84 [=====] - 0s 5ms/step - loss: 0.0036
```

7. Visualize encoder latent spaces

The encoder maps each 600-dimensional input window to a 32-dimensional latent representation. The t-SNE plots below visualize these latent spaces.

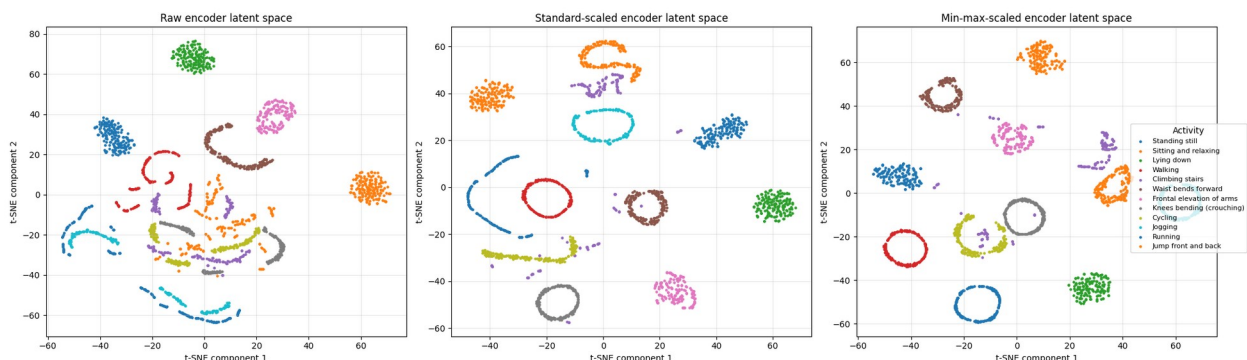
```
def plot_latent_tsne(encoder, X_test, y_test, title, ax):
    X_sub, y_sub = stratified_subset(X_test, y_test)
    latent = encoder.predict(X_sub, verbose=0)
    latent_tsne = compute_tsne(latent)

    for label in sorted(np.unique(y_sub)):
        idx = y_sub == label
        ax.scatter(latent_tsne[idx, 0], latent_tsne[idx, 1], s=5,
label=label_map[label])

    ax.set_title(title)
    ax.set_xlabel("t-SNE component 1")
    ax.set_ylabel("t-SNE component 2")
    ax.grid(True, alpha=0.3)

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
plot_latent_tsne(models["raw"][1], X_test_raw, y_test_raw, "Raw
encoder latent space", axes[0])
plot_latent_tsne(models["std"][1], X_test_std, y_test_std, "Standard-
scaled encoder latent space", axes[1])
plot_latent_tsne(models["minmax"][1], X_test_minmax, y_test_minmax,
"Min-max-scaled encoder latent space", axes[2])

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, title="Activity", loc="center right",
bbox_to_anchor=(1.02, 0.5), fontsize=8)
plt.tight_layout()
plt.show()
```



8. Train one classifier on each latent representation

Each classifier receives a 32-dimensional encoder output and predicts one of 12 activity classes.

Classifier architecture for each branch:

32 latent input neurons -> Dense(20, ReLU) -> Dense(12, Softmax)

```
from tensorflow.keras.utils import to_categorical

CLASSIFIER_EPOCHS = 50

y_train_raw_cat = to_categorical(y_train_raw, num_classes=NUM_CLASSES)
y_test_raw_cat = to_categorical(y_test_raw, num_classes=NUM_CLASSES)
y_train_std_cat = to_categorical(y_train_std, num_classes=NUM_CLASSES)
y_test_std_cat = to_categorical(y_test_std, num_classes=NUM_CLASSES)
y_train_minmax_cat = to_categorical(y_train_minmax,
num_classes=NUM_CLASSES)
y_test_minmax_cat = to_categorical(y_test_minmax,
num_classes=NUM_CLASSES)

latent_train_raw = models["raw"][1].predict(X_train_raw, verbose=0)
latent_test_raw = models["raw"][1].predict(X_test_raw, verbose=0)
latent_train_std = models["std"][1].predict(X_train_std, verbose=0)
latent_test_std = models["std"][1].predict(X_test_std, verbose=0)
latent_train_minmax = models["minmax"][1].predict(X_train_minmax,
verbose=0)
latent_test_minmax = models["minmax"][1].predict(X_test_minmax,
verbose=0)

def build_classifier(input_dim, num_classes=NUM_CLASSES):
    model = Sequential([
        Input(shape=(input_dim,), name="latent_input"),
        Dense(20, activation="relu", name="classifier_dense_20"),
        Dense(num_classes, activation="softmax",
name="activity_softmax"),
    ], name="latent_classifier")
    model.compile(optimizer=Adam(learning_rate=0.001),
loss="categorical_crossentropy", metrics=["accuracy"])
    return model

clf_raw = build_classifier(latent_train_raw.shape[1])
clf_std = build_classifier(latent_train_std.shape[1])
clf_minmax = build_classifier(latent_train_minmax.shape[1])

print("Training classifier for raw branch")
clf_raw.fit(latent_train_raw, y_train_raw_cat,
epochs=CLASSIFIER_EPOCHS, batch_size=BATCH_SIZE, verbose=1)

print("Training classifier for standard-scaled branch")
clf_std.fit(latent_train_std, y_train_std_cat,
epochs=CLASSIFIER_EPOCHS, batch_size=BATCH_SIZE, verbose=1)

print("Training classifier for min-max-scaled branch")
```

```
clf_minmax.fit(latent_train_minmax, y_train_minmax_cat,  
epochs=CLASSIFIER_EPOCHS, batch_size=BATCH_SIZE, verbose=1)
```

Training classifier for raw branch

Epoch 1/50

84/84 [=====] - 1s 2ms/step - loss: 14.0139 -
accuracy: 0.1600

Epoch 2/50

84/84 [=====] - 0s 3ms/step - loss: 4.1432 -
accuracy: 0.2825

Epoch 3/50

84/84 [=====] - 0s 3ms/step - loss: 2.2622 -
accuracy: 0.4164

Epoch 4/50

84/84 [=====] - 0s 3ms/step - loss: 1.4383 -
accuracy: 0.5746

Epoch 5/50

84/84 [=====] - 0s 2ms/step - loss: 1.0523 -
accuracy: 0.6715

Epoch 6/50

84/84 [=====] - 0s 3ms/step - loss: 0.8310 -
accuracy: 0.7296

Epoch 7/50

84/84 [=====] - 0s 3ms/step - loss: 0.6808 -
accuracy: 0.7731

Epoch 8/50

84/84 [=====] - 0s 4ms/step - loss: 0.5737 -
accuracy: 0.8053

Epoch 9/50

84/84 [=====] - 0s 3ms/step - loss: 0.4990 -
accuracy: 0.8293

Epoch 10/50

84/84 [=====] - 0s 3ms/step - loss: 0.4380 -
accuracy: 0.8462

Epoch 11/50

84/84 [=====] - 0s 3ms/step - loss: 0.3648 -
accuracy: 0.8750

Epoch 12/50

84/84 [=====] - 0s 3ms/step - loss: 0.3201 -
accuracy: 0.8933

Epoch 13/50

84/84 [=====] - 0s 2ms/step - loss: 0.2822 -
accuracy: 0.9144

Epoch 14/50

84/84 [=====] - 0s 1ms/step - loss: 0.2478 -
accuracy: 0.9313

Epoch 15/50

84/84 [=====] - 0s 1ms/step - loss: 0.2229 -
accuracy: 0.9420

Epoch 16/50

```
84/84 [=====] - 0s 1ms/step - loss: 0.2020 -  
accuracy: 0.9490  
Epoch 17/50  
84/84 [=====] - 0s 1ms/step - loss: 0.1839 -  
accuracy: 0.9566  
Epoch 18/50  
84/84 [=====] - 0s 2ms/step - loss: 0.1678 -  
accuracy: 0.9615  
Epoch 19/50  
84/84 [=====] - 0s 2ms/step - loss: 0.1517 -  
accuracy: 0.9664  
Epoch 20/50  
84/84 [=====] - 0s 2ms/step - loss: 0.1370 -  
accuracy: 0.9713  
Epoch 21/50  
84/84 [=====] - 0s 1ms/step - loss: 0.1255 -  
accuracy: 0.9747  
Epoch 22/50  
84/84 [=====] - 0s 2ms/step - loss: 0.1169 -  
accuracy: 0.9758  
Epoch 23/50  
84/84 [=====] - 0s 4ms/step - loss: 0.1090 -  
accuracy: 0.9778  
Epoch 24/50  
84/84 [=====] - 0s 3ms/step - loss: 0.1005 -  
accuracy: 0.9789  
Epoch 25/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0931 -  
accuracy: 0.9820  
Epoch 26/50  
84/84 [=====] - 0s 4ms/step - loss: 0.0825 -  
accuracy: 0.9849  
Epoch 27/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0768 -  
accuracy: 0.9856  
Epoch 28/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0688 -  
accuracy: 0.9865  
Epoch 29/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0649 -  
accuracy: 0.9869  
Epoch 30/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0607 -  
accuracy: 0.9881  
Epoch 31/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0577 -  
accuracy: 0.9889  
Epoch 32/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0541 -
```

```
accuracy: 0.9890
Epoch 33/50
84/84 [=====] - 0s 1ms/step - loss: 0.0510 -
accuracy: 0.9902
Epoch 34/50
84/84 [=====] - 0s 1ms/step - loss: 0.0490 -
accuracy: 0.9899
Epoch 35/50
84/84 [=====] - 3s 38ms/step - loss: 0.0467 -
accuracy: 0.9904
Epoch 36/50
84/84 [=====] - 0s 2ms/step - loss: 0.0431 -
accuracy: 0.9916
Epoch 37/50
84/84 [=====] - 0s 2ms/step - loss: 0.0416 -
accuracy: 0.9919
Epoch 38/50
84/84 [=====] - 0s 1ms/step - loss: 0.0405 -
accuracy: 0.9917
Epoch 39/50
84/84 [=====] - 0s 2ms/step - loss: 0.0389 -
accuracy: 0.9925
Epoch 40/50
84/84 [=====] - 0s 2ms/step - loss: 0.0364 -
accuracy: 0.9929
Epoch 41/50
84/84 [=====] - 0s 2ms/step - loss: 0.0358 -
accuracy: 0.9930
Epoch 42/50
84/84 [=====] - 0s 2ms/step - loss: 0.0333 -
accuracy: 0.9940
Epoch 43/50
84/84 [=====] - 0s 2ms/step - loss: 0.0340 -
accuracy: 0.9933
Epoch 44/50
84/84 [=====] - 0s 2ms/step - loss: 0.0322 -
accuracy: 0.9941
Epoch 45/50
84/84 [=====] - 0s 2ms/step - loss: 0.0299 -
accuracy: 0.9943
Epoch 46/50
84/84 [=====] - 0s 2ms/step - loss: 0.0284 -
accuracy: 0.9948
Epoch 47/50
84/84 [=====] - 0s 1ms/step - loss: 0.0280 -
accuracy: 0.9947
Epoch 48/50
84/84 [=====] - 0s 2ms/step - loss: 0.0265 -
accuracy: 0.9953
```



```
Epoch 49/50
84/84 [=====] - 0s 1ms/step - loss: 0.0257 -
accuracy: 0.9954
Epoch 50/50
84/84 [=====] - 0s 1ms/step - loss: 0.0249 -
accuracy: 0.9949
Training classifier for standard-scaled branch
Epoch 1/50
84/84 [=====] - 1s 3ms/step - loss: 2.6563 -
accuracy: 0.3243
Epoch 2/50
84/84 [=====] - 0s 2ms/step - loss: 0.9916 -
accuracy: 0.7087
Epoch 3/50
84/84 [=====] - 0s 4ms/step - loss: 0.5710 -
accuracy: 0.8801
Epoch 4/50
84/84 [=====] - 0s 3ms/step - loss: 0.3774 -
accuracy: 0.9311
Epoch 5/50
84/84 [=====] - 0s 4ms/step - loss: 0.2726 -
accuracy: 0.9611
Epoch 6/50
84/84 [=====] - 0s 3ms/step - loss: 0.2089 -
accuracy: 0.9758
Epoch 7/50
84/84 [=====] - 0s 2ms/step - loss: 0.1646 -
accuracy: 0.9810
Epoch 8/50
84/84 [=====] - 0s 2ms/step - loss: 0.1326 -
accuracy: 0.9857
Epoch 9/50
84/84 [=====] - 0s 2ms/step - loss: 0.1083 -
accuracy: 0.9887
Epoch 10/50
84/84 [=====] - 0s 2ms/step - loss: 0.0896 -
accuracy: 0.9908
Epoch 11/50
84/84 [=====] - 0s 2ms/step - loss: 0.0748 -
accuracy: 0.9924
Epoch 12/50
84/84 [=====] - 0s 2ms/step - loss: 0.0631 -
accuracy: 0.9936
Epoch 13/50
84/84 [=====] - 0s 1ms/step - loss: 0.0538 -
accuracy: 0.9949
Epoch 14/50
84/84 [=====] - 0s 2ms/step - loss: 0.0460 -
accuracy: 0.9963
```

```
Epoch 15/50
84/84 [=====] - 0s 3ms/step - loss: 0.0400 -
accuracy: 0.9971
Epoch 16/50
84/84 [=====] - 0s 3ms/step - loss: 0.0348 -
accuracy: 0.9977
Epoch 17/50
84/84 [=====] - 0s 3ms/step - loss: 0.0305 -
accuracy: 0.9984
Epoch 18/50
84/84 [=====] - 0s 3ms/step - loss: 0.0269 -
accuracy: 0.9986
Epoch 19/50
84/84 [=====] - 0s 3ms/step - loss: 0.0238 -
accuracy: 0.9988
Epoch 20/50
84/84 [=====] - 0s 2ms/step - loss: 0.0212 -
accuracy: 0.9988
Epoch 21/50
84/84 [=====] - 0s 2ms/step - loss: 0.0190 -
accuracy: 0.9992
Epoch 22/50
84/84 [=====] - 0s 3ms/step - loss: 0.0170 -
accuracy: 0.9993
Epoch 23/50
84/84 [=====] - 0s 2ms/step - loss: 0.0153 -
accuracy: 0.9995
Epoch 24/50
84/84 [=====] - 0s 2ms/step - loss: 0.0139 -
accuracy: 0.9994
Epoch 25/50
84/84 [=====] - 0s 2ms/step - loss: 0.0126 -
accuracy: 0.9997
Epoch 26/50
84/84 [=====] - 0s 1ms/step - loss: 0.0115 -
accuracy: 0.9996
Epoch 27/50
84/84 [=====] - 0s 1ms/step - loss: 0.0105 -
accuracy: 0.9997
Epoch 28/50
84/84 [=====] - 0s 2ms/step - loss: 0.0096 -
accuracy: 0.9998
Epoch 29/50
84/84 [=====] - 0s 3ms/step - loss: 0.0088 -
accuracy: 0.9998
Epoch 30/50
84/84 [=====] - 0s 2ms/step - loss: 0.0082 -
accuracy: 0.9998
Epoch 31/50
```

```
84/84 [=====] - 0s 1ms/step - loss: 0.0076 -  
accuracy: 0.9999  
Epoch 32/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0069 -  
accuracy: 0.9998  
Epoch 33/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0065 -  
accuracy: 0.9999  
Epoch 34/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0060 -  
accuracy: 0.9999  
Epoch 35/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0056 -  
accuracy: 0.9999  
Epoch 36/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0052 -  
accuracy: 0.9999  
Epoch 37/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0049 -  
accuracy: 0.9999  
Epoch 38/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0045 -  
accuracy: 1.0000  
Epoch 39/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0043 -  
accuracy: 0.9999  
Epoch 40/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0040 -  
accuracy: 0.9999  
Epoch 41/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0037 -  
accuracy: 0.9999  
Epoch 42/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0035 -  
accuracy: 0.9999  
Epoch 43/50  
84/84 [=====] - 0s 1ms/step - loss: 0.0033 -  
accuracy: 1.0000  
Epoch 44/50  
84/84 [=====] - 0s 3ms/step - loss: 0.0031 -  
accuracy: 0.9999  
Epoch 45/50  
84/84 [=====] - 0s 4ms/step - loss: 0.0029 -  
accuracy: 1.0000  
Epoch 46/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0027 -  
accuracy: 1.0000  
Epoch 47/50  
84/84 [=====] - 0s 2ms/step - loss: 0.0026 -
```

```
accuracy: 1.0000
Epoch 48/50
84/84 [=====] - 0s 2ms/step - loss: 0.0024 -
accuracy: 1.0000
Epoch 49/50
84/84 [=====] - 0s 2ms/step - loss: 0.0023 -
accuracy: 1.0000
Epoch 50/50
84/84 [=====] - 0s 3ms/step - loss: 0.0021 -
accuracy: 1.0000
Training classifier for min-max-scaled branch
Epoch 1/50
84/84 [=====] - 1s 3ms/step - loss: 2.2898 -
accuracy: 0.2053
Epoch 2/50
84/84 [=====] - 0s 2ms/step - loss: 1.7763 -
accuracy: 0.4260
Epoch 3/50
84/84 [=====] - 0s 1ms/step - loss: 1.3924 -
accuracy: 0.6256
Epoch 4/50
84/84 [=====] - 0s 1ms/step - loss: 1.1367 -
accuracy: 0.6896
Epoch 5/50
84/84 [=====] - 0s 1ms/step - loss: 0.9515 -
accuracy: 0.7296
Epoch 6/50
84/84 [=====] - 0s 2ms/step - loss: 0.8145 -
accuracy: 0.7629
Epoch 7/50
84/84 [=====] - 0s 2ms/step - loss: 0.7111 -
accuracy: 0.8052
Epoch 8/50
84/84 [=====] - 0s 2ms/step - loss: 0.6302 -
accuracy: 0.8422
Epoch 9/50
84/84 [=====] - 0s 3ms/step - loss: 0.5621 -
accuracy: 0.8565
Epoch 10/50
84/84 [=====] - 0s 5ms/step - loss: 0.5049 -
accuracy: 0.8795
Epoch 11/50
84/84 [=====] - 0s 2ms/step - loss: 0.4559 -
accuracy: 0.9043
Epoch 12/50
84/84 [=====] - 0s 2ms/step - loss: 0.4127 -
accuracy: 0.9207
Epoch 13/50
84/84 [=====] - 0s 2ms/step - loss: 0.3752 -
```

```
accuracy: 0.9308
Epoch 14/50
84/84 [=====] - 0s 1ms/step - loss: 0.3416 -
accuracy: 0.9390
Epoch 15/50
84/84 [=====] - 0s 2ms/step - loss: 0.3140 -
accuracy: 0.9434
Epoch 16/50
84/84 [=====] - 0s 4ms/step - loss: 0.2901 -
accuracy: 0.9490
Epoch 17/50
84/84 [=====] - 0s 3ms/step - loss: 0.2691 -
accuracy: 0.9521
Epoch 18/50
84/84 [=====] - 0s 1ms/step - loss: 0.2502 -
accuracy: 0.9549
Epoch 19/50
84/84 [=====] - 0s 1ms/step - loss: 0.2336 -
accuracy: 0.9572
Epoch 20/50
84/84 [=====] - 0s 1ms/step - loss: 0.2189 -
accuracy: 0.9605
Epoch 21/50
84/84 [=====] - 0s 1ms/step - loss: 0.2052 -
accuracy: 0.9622
Epoch 22/50
84/84 [=====] - 0s 2ms/step - loss: 0.1924 -
accuracy: 0.9657
Epoch 23/50
84/84 [=====] - 0s 3ms/step - loss: 0.1807 -
accuracy: 0.9692
Epoch 24/50
84/84 [=====] - 0s 3ms/step - loss: 0.1697 -
accuracy: 0.9718
Epoch 25/50
84/84 [=====] - 0s 3ms/step - loss: 0.1602 -
accuracy: 0.9740
Epoch 26/50
84/84 [=====] - 0s 4ms/step - loss: 0.1507 -
accuracy: 0.9764
Epoch 27/50
84/84 [=====] - 0s 4ms/step - loss: 0.1424 -
accuracy: 0.9776
Epoch 28/50
84/84 [=====] - 0s 5ms/step - loss: 0.1343 -
accuracy: 0.9803
Epoch 29/50
84/84 [=====] - 0s 3ms/step - loss: 0.1269 -
accuracy: 0.9818
```

```
Epoch 30/50
84/84 [=====] - 0s 4ms/step - loss: 0.1202 -
accuracy: 0.9828
Epoch 31/50
84/84 [=====] - 0s 2ms/step - loss: 0.1141 -
accuracy: 0.9839
Epoch 32/50
84/84 [=====] - 0s 1ms/step - loss: 0.1081 -
accuracy: 0.9852
Epoch 33/50
84/84 [=====] - 0s 2ms/step - loss: 0.1028 -
accuracy: 0.9855
Epoch 34/50
84/84 [=====] - 0s 4ms/step - loss: 0.0973 -
accuracy: 0.9872
Epoch 35/50
84/84 [=====] - 0s 2ms/step - loss: 0.0931 -
accuracy: 0.9866
Epoch 36/50
84/84 [=====] - 0s 1ms/step - loss: 0.0877 -
accuracy: 0.9880
Epoch 37/50
84/84 [=====] - 0s 1ms/step - loss: 0.0839 -
accuracy: 0.9884
Epoch 38/50
84/84 [=====] - 0s 1ms/step - loss: 0.0798 -
accuracy: 0.9892
Epoch 39/50
84/84 [=====] - 0s 1ms/step - loss: 0.0761 -
accuracy: 0.9896
Epoch 40/50
84/84 [=====] - 0s 2ms/step - loss: 0.0728 -
accuracy: 0.9899
Epoch 41/50
84/84 [=====] - 0s 4ms/step - loss: 0.0696 -
accuracy: 0.9907
Epoch 42/50
84/84 [=====] - 0s 3ms/step - loss: 0.0668 -
accuracy: 0.9905
Epoch 43/50
84/84 [=====] - 0s 4ms/step - loss: 0.0638 -
accuracy: 0.9915
Epoch 44/50
84/84 [=====] - 0s 4ms/step - loss: 0.0612 -
accuracy: 0.9918
Epoch 45/50
84/84 [=====] - 0s 4ms/step - loss: 0.0585 -
accuracy: 0.9919
Epoch 46/50
```

```

84/84 [=====] - 0s 3ms/step - loss: 0.0565 -
accuracy: 0.9923
Epoch 47/50
84/84 [=====] - 0s 4ms/step - loss: 0.0541 -
accuracy: 0.9927
Epoch 48/50
84/84 [=====] - 0s 3ms/step - loss: 0.0522 -
accuracy: 0.9926
Epoch 49/50
84/84 [=====] - 0s 5ms/step - loss: 0.0499 -
accuracy: 0.9931
Epoch 50/50
84/84 [=====] - 0s 1ms/step - loss: 0.0482 -
accuracy: 0.9934

<keras.src.callbacks.History at 0x7b80ac0bf090>

```

9. Evaluate the three branch classifiers

The branch classifiers are evaluated separately before constructing the stacked ensemble.

```

from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

def evaluate_classifier(model, X_test, y_test_true, title):
    y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)
    target_names = [label_map[i] for i in range(NUM_CLASSES)]

    print(f"\nClassification report: {title}")
    print(classification_report(y_test_true, y_pred,
target_names=target_names, zero_division=0))

    cm = confusion_matrix(y_test_true, y_pred,
labels=list(range(NUM_CLASSES)))
    fig, ax = plt.subplots(figsize=(10, 8))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=target_names)
    disp.plot(ax=ax, xticks_rotation=45, values_format="d",
colorbar=False)
    ax.set_title(f"Confusion matrix: {title}")
    plt.tight_layout()
    plt.show()

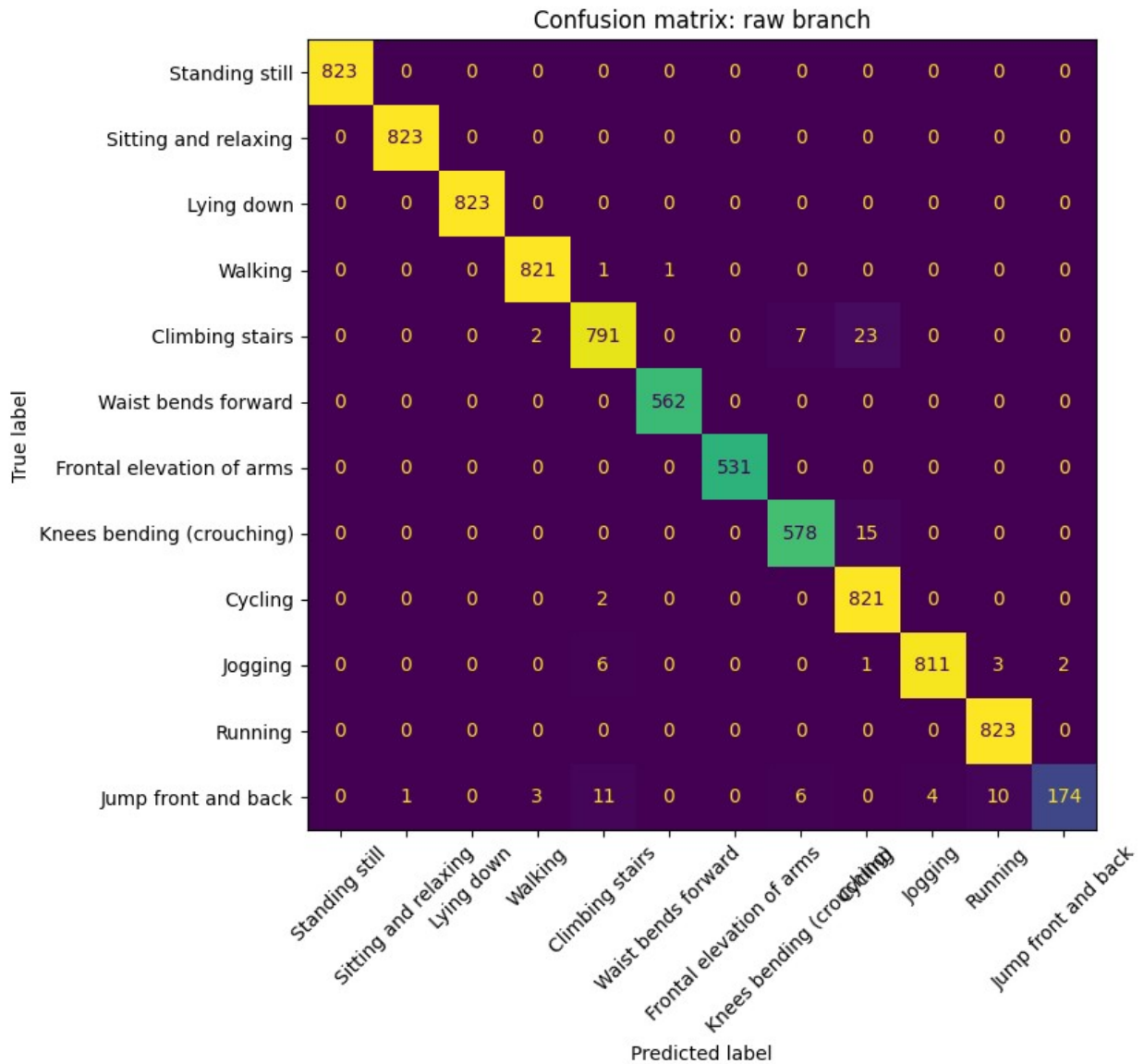
evaluate_classifier(clf_raw, latent_test_raw, y_test_raw, "raw
branch")
evaluate_classifier(clf_std, latent_test_std, y_test_std, "standard-
scaled branch")

```

```
evaluate_classifier(clf_minmax, latent_test_minmax, y_test_minmax,  
"min-max-scaled branch")
```

Classification report: raw branch

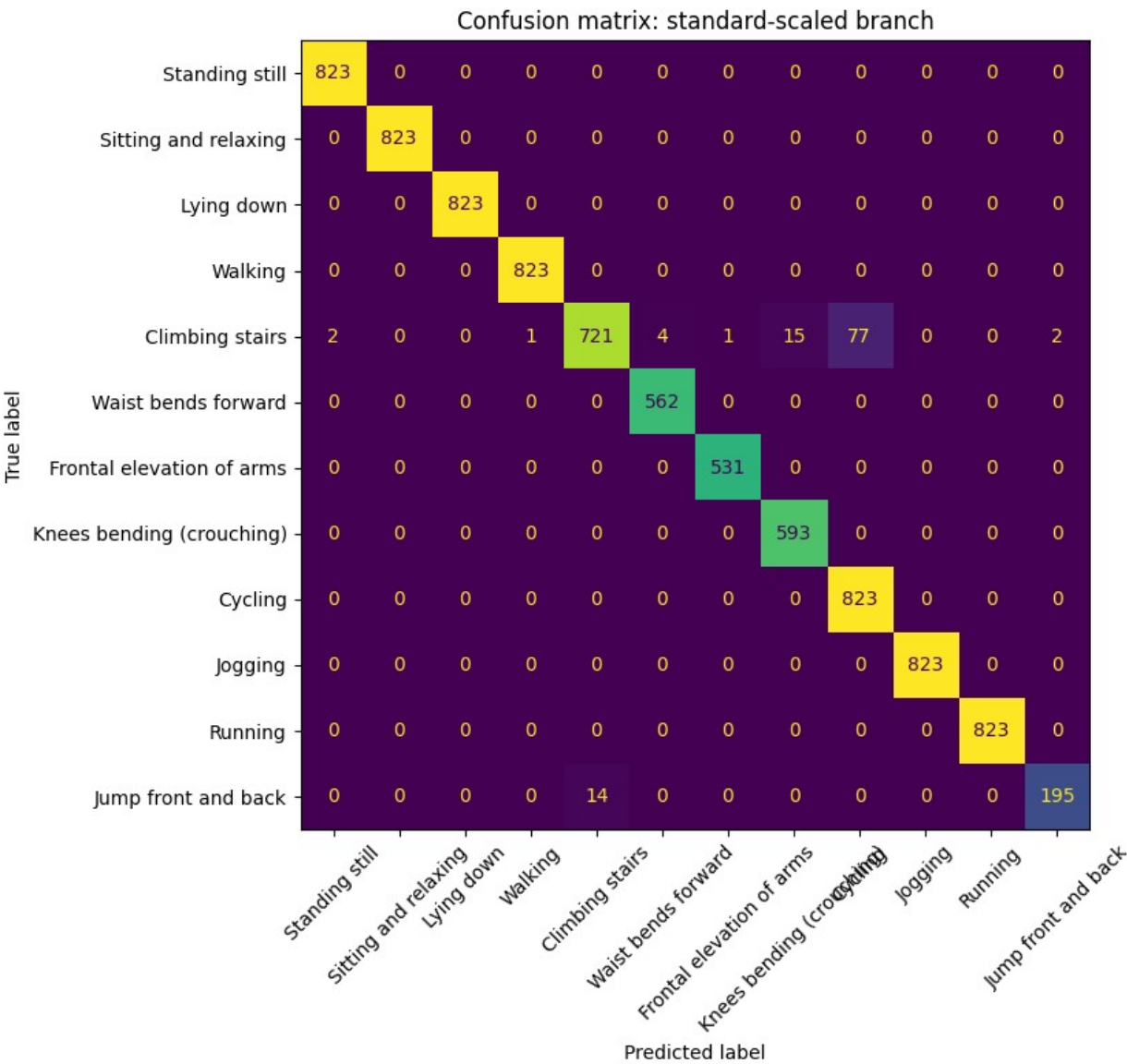
	precision	recall	f1-score	support
Standing still	1.00	1.00	1.00	823
Sitting and relaxing	1.00	1.00	1.00	823
Lying down	1.00	1.00	1.00	823
Walking	0.99	1.00	1.00	823
Climbing stairs	0.98	0.96	0.97	823
Waist bends forward	1.00	1.00	1.00	562
Frontal elevation of arms	1.00	1.00	1.00	531
Knees bending (crouching)	0.98	0.97	0.98	593
Cycling	0.95	1.00	0.98	823
Jogging	1.00	0.99	0.99	823
Running	0.98	1.00	0.99	823
Jump front and back	0.99	0.83	0.90	209
accuracy			0.99	8479
macro avg	0.99	0.98	0.98	8479
weighted avg	0.99	0.99	0.99	8479



Classification report: standard-scaled branch

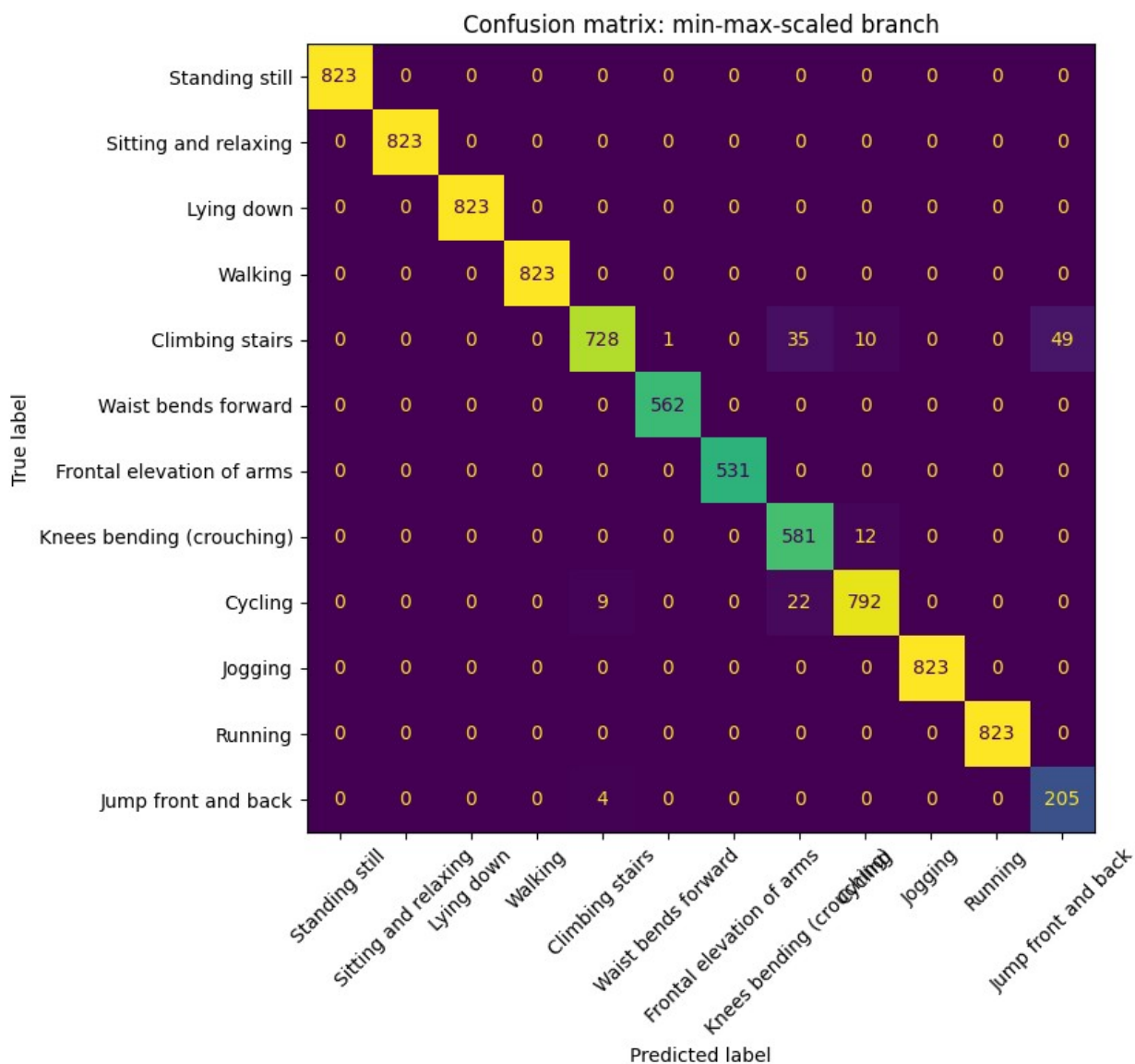
	precision	recall	f1-score	support
Standing still	1.00	1.00	1.00	823
Sitting and relaxing	1.00	1.00	1.00	823
Lying down	1.00	1.00	1.00	823
Walking	1.00	1.00	1.00	823
Climbing stairs	0.98	0.88	0.93	823
Waist bends forward	0.99	1.00	1.00	562
Frontal elevation of arms	1.00	1.00	1.00	531
Knees bending (crouching)	0.98	1.00	0.99	593
Cycling	0.91	1.00	0.96	823
Jogging	1.00	1.00	1.00	823
Running	1.00	1.00	1.00	823

Jump front and back	0.99	0.93	0.96	209
accuracy			0.99	8479
macro avg	0.99	0.98	0.99	8479
weighted avg	0.99	0.99	0.99	8479



Classification report: min-max-scaled branch				
	precision	recall	f1-score	support
Standing still	1.00	1.00	1.00	823
Sitting and relaxing	1.00	1.00	1.00	823
Lying down	1.00	1.00	1.00	823

Walking	1.00	1.00	1.00	823
Climbing stairs	0.98	0.88	0.93	823
Waist bends forward	1.00	1.00	1.00	562
Frontal elevation of arms	1.00	1.00	1.00	531
Knees bending (crouching)	0.91	0.98	0.94	593
Cycling	0.97	0.96	0.97	823
Jogging	1.00	1.00	1.00	823
Running	1.00	1.00	1.00	823
Jump front and back	0.81	0.98	0.89	209
accuracy			0.98	8479
macro avg	0.97	0.98	0.98	8479
weighted avg	0.98	0.98	0.98	8479



10. Build the stacked ensemble

The three branch classifiers each output a 12-dimensional softmax vector. These outputs are concatenated into a 36-dimensional vector:

12 raw probabilities + 12 standard-scaled probabilities + 12 min-max-scaled probabilities = 36 ensemble inputs

The stacked meta-classifier then maps the 36-dimensional ensemble input to the final 12-class prediction.

Meta-classifier architecture:

36 input neurons -> Dense(24, ReLU) -> Dense(12, Softmax)

```
train_logits_raw = clf_raw.predict(latent_train_raw, verbose=0)
test_logits_raw = clf_raw.predict(latent_test_raw, verbose=0)

train_logits_std = clf_std.predict(latent_train_std, verbose=0)
test_logits_std = clf_std.predict(latent_test_std, verbose=0)

train_logits_minmax = clf_minmax.predict(latent_train_minmax,
verbose=0)
test_logits_minmax = clf_minmax.predict(latent_test_minmax, verbose=0)

X_train_ensemble = np.hstack([train_logits_raw, train_logits_std,
train_logits_minmax]).astype(np.float32)
X_test_ensemble = np.hstack([test_logits_raw, test_logits_std,
test_logits_minmax]).astype(np.float32)

y_train_ensemble_cat = to_categorical(y_train_raw,
num_classes=NUM_CLASSES)
y_test_ensemble_cat = to_categorical(y_test_raw,
num_classes=NUM_CLASSES)

meta_clf = Sequential([
    Input(shape=(36,), name="ensemble_input"),
    Dense(24, activation="relu", name="meta_dense_24"),
    Dense(NUM_CLASSES, activation="softmax", name="meta_softmax"),
], name="stacked_meta_classifier")

meta_clf.compile(optimizer=Adam(learning_rate=0.001),
loss="categorical_crossentropy", metrics=["accuracy"])
meta_clf.fit(X_train_ensemble, y_train_ensemble_cat,
epochs=CLASSIFIER_EPOCHS, batch_size=BATCH_SIZE, verbose=1)

evaluate_classifier(meta_clf, X_test_ensemble, y_test_raw, "stacked
ensemble")
```

Epoch 1/50

84/84 [=====] - 0s 2ms/step - loss: 2.0397 -

```
accuracy: 0.6092
Epoch 2/50
84/84 [=====] - 0s 2ms/step - loss: 1.0955 -
accuracy: 0.9995
Epoch 3/50
84/84 [=====] - 0s 3ms/step - loss: 0.3665 -
accuracy: 1.0000
Epoch 4/50
84/84 [=====] - 0s 1ms/step - loss: 0.1233 -
accuracy: 1.0000
Epoch 5/50
84/84 [=====] - 0s 1ms/step - loss: 0.0573 -
accuracy: 1.0000
Epoch 6/50
84/84 [=====] - 0s 1ms/step - loss: 0.0333 -
accuracy: 1.0000
Epoch 7/50
84/84 [=====] - 3s 37ms/step - loss: 0.0214 -
accuracy: 1.0000
Epoch 8/50
84/84 [=====] - 0s 2ms/step - loss: 0.0148 -
accuracy: 1.0000
Epoch 9/50
84/84 [=====] - 0s 3ms/step - loss: 0.0109 -
accuracy: 1.0000
Epoch 10/50
84/84 [=====] - 0s 3ms/step - loss: 0.0083 -
accuracy: 1.0000
Epoch 11/50
84/84 [=====] - 0s 3ms/step - loss: 0.0066 -
accuracy: 1.0000
Epoch 12/50
84/84 [=====] - 0s 3ms/step - loss: 0.0054 -
accuracy: 1.0000
Epoch 13/50
84/84 [=====] - 0s 2ms/step - loss: 0.0044 -
accuracy: 1.0000
Epoch 14/50
84/84 [=====] - 0s 2ms/step - loss: 0.0037 -
accuracy: 1.0000
Epoch 15/50
84/84 [=====] - 0s 3ms/step - loss: 0.0032 -
accuracy: 1.0000
Epoch 16/50
84/84 [=====] - 0s 2ms/step - loss: 0.0028 -
accuracy: 1.0000
Epoch 17/50
84/84 [=====] - 0s 3ms/step - loss: 0.0024 -
accuracy: 1.0000
```

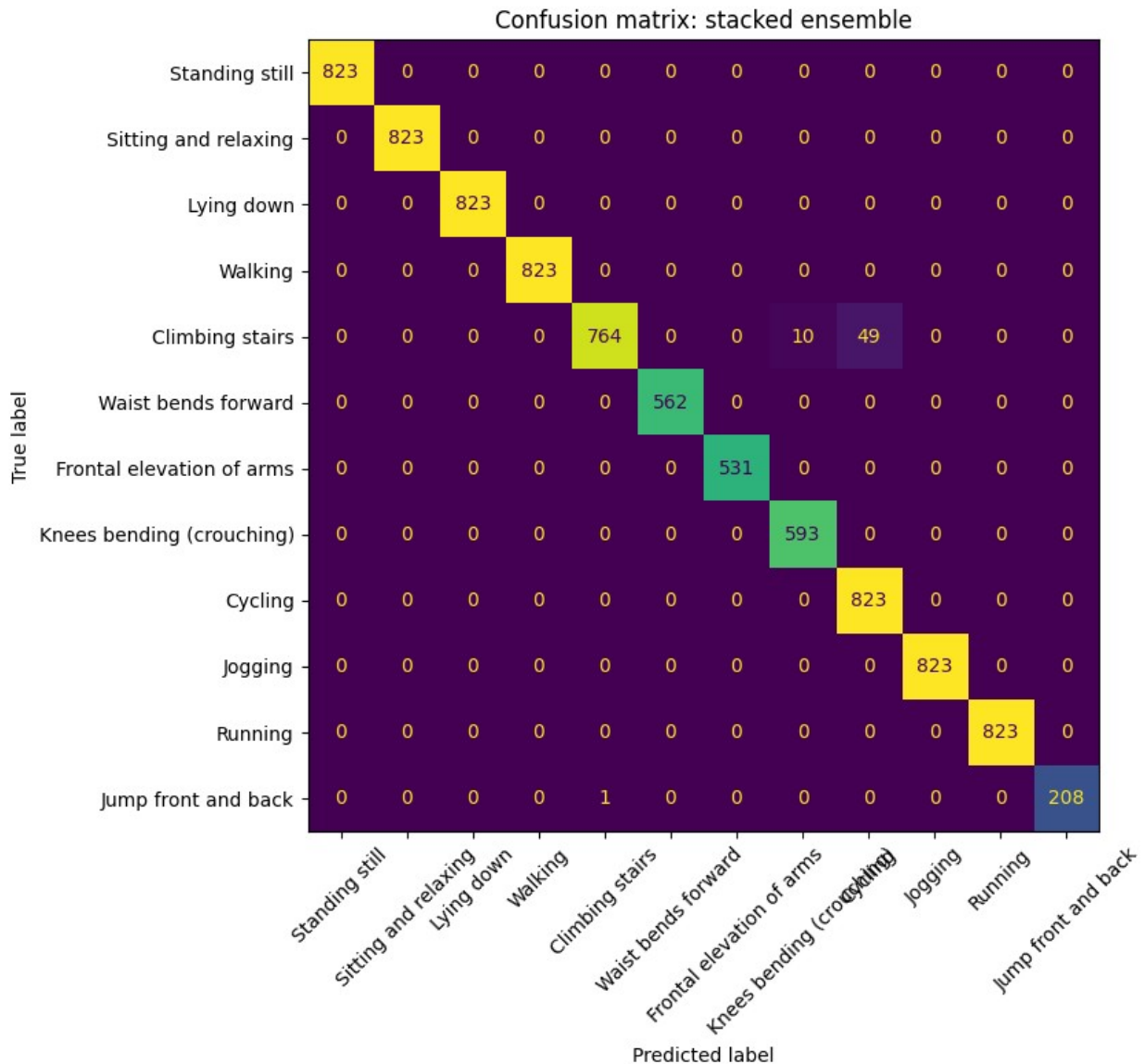
```
Epoch 18/50
84/84 [=====] - 0s 3ms/step - loss: 0.0021 -
accuracy: 1.0000
Epoch 19/50
84/84 [=====] - 0s 1ms/step - loss: 0.0019 -
accuracy: 1.0000
Epoch 20/50
84/84 [=====] - 0s 1ms/step - loss: 0.0017 -
accuracy: 1.0000
Epoch 21/50
84/84 [=====] - 0s 1ms/step - loss: 0.0015 -
accuracy: 1.0000
Epoch 22/50
84/84 [=====] - 0s 1ms/step - loss: 0.0013 -
accuracy: 1.0000
Epoch 23/50
84/84 [=====] - 0s 2ms/step - loss: 0.0012 -
accuracy: 1.0000
Epoch 24/50
84/84 [=====] - 0s 3ms/step - loss: 0.0011 -
accuracy: 1.0000
Epoch 25/50
84/84 [=====] - 0s 3ms/step - loss: 9.9467e-
04 - accuracy: 1.0000
Epoch 26/50
84/84 [=====] - 0s 3ms/step - loss: 9.0794e-
04 - accuracy: 1.0000
Epoch 27/50
84/84 [=====] - 0s 2ms/step - loss: 8.3104e-
04 - accuracy: 1.0000
Epoch 28/50
84/84 [=====] - 0s 4ms/step - loss: 7.6269e-
04 - accuracy: 1.0000
Epoch 29/50
84/84 [=====] - 0s 4ms/step - loss: 7.0170e-
04 - accuracy: 1.0000
Epoch 30/50
84/84 [=====] - 0s 2ms/step - loss: 6.4700e-
04 - accuracy: 1.0000
Epoch 31/50
84/84 [=====] - 0s 3ms/step - loss: 5.9781e-
04 - accuracy: 1.0000
Epoch 32/50
84/84 [=====] - 0s 3ms/step - loss: 5.5289e-
04 - accuracy: 1.0000
Epoch 33/50
84/84 [=====] - 0s 2ms/step - loss: 5.1200e-
04 - accuracy: 1.0000
Epoch 34/50
```

```
84/84 [=====] - 0s 1ms/step - loss: 4.7513e-
04 - accuracy: 1.0000
Epoch 35/50
84/84 [=====] - 0s 1ms/step - loss: 4.4179e-
04 - accuracy: 1.0000
Epoch 36/50
84/84 [=====] - 0s 1ms/step - loss: 4.1141e-
04 - accuracy: 1.0000
Epoch 37/50
84/84 [=====] - 0s 1ms/step - loss: 3.8367e-
04 - accuracy: 1.0000
Epoch 38/50
84/84 [=====] - 0s 2ms/step - loss: 3.5828e-
04 - accuracy: 1.0000
Epoch 39/50
84/84 [=====] - 0s 2ms/step - loss: 3.3500e-
04 - accuracy: 1.0000
Epoch 40/50
84/84 [=====] - 0s 2ms/step - loss: 3.1355e-
04 - accuracy: 1.0000
Epoch 41/50
84/84 [=====] - 0s 3ms/step - loss: 2.9378e-
04 - accuracy: 1.0000
Epoch 42/50
84/84 [=====] - 0s 3ms/step - loss: 2.7555e-
04 - accuracy: 1.0000
Epoch 43/50
84/84 [=====] - 0s 3ms/step - loss: 2.5865e-
04 - accuracy: 1.0000
Epoch 44/50
84/84 [=====] - 0s 2ms/step - loss: 2.4300e-
04 - accuracy: 1.0000
Epoch 45/50
84/84 [=====] - 0s 3ms/step - loss: 2.2848e-
04 - accuracy: 1.0000
Epoch 46/50
84/84 [=====] - 0s 2ms/step - loss: 2.1500e-
04 - accuracy: 1.0000
Epoch 47/50
84/84 [=====] - 0s 2ms/step - loss: 2.0248e-
04 - accuracy: 1.0000
Epoch 48/50
84/84 [=====] - 0s 3ms/step - loss: 1.9080e-
04 - accuracy: 1.0000
Epoch 49/50
84/84 [=====] - 0s 1ms/step - loss: 1.7988e-
04 - accuracy: 1.0000
Epoch 50/50
84/84 [=====] - 0s 1ms/step - loss: 1.6970e-
```

04 - accuracy: 1.0000

Classification report: stacked ensemble

	precision	recall	f1-score	support
Standing still	1.00	1.00	1.00	823
Sitting and relaxing	1.00	1.00	1.00	823
Lying down	1.00	1.00	1.00	823
Walking	1.00	1.00	1.00	823
Climbing stairs	1.00	0.93	0.96	823
Waist bends forward	1.00	1.00	1.00	562
Frontal elevation of arms	1.00	1.00	1.00	531
Knees bending (crouching)	0.98	1.00	0.99	593
Cycling	0.94	1.00	0.97	823
Jogging	1.00	1.00	1.00	823
Running	1.00	1.00	1.00	823
Jump front and back	1.00	1.00	1.00	209
accuracy			0.99	8479
macro avg	0.99	0.99	0.99	8479
weighted avg	0.99	0.99	0.99	8479



11. Convert the trained models to baseline TFLite models

This section converts the trained encoder, branch classifier, and meta-classifier models to regular TFLite models and reports their file sizes. These are not yet the pruned QAT int8 models.

```
def convert_and_save_tflite(model, name):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    tflite_model = converter.convert()

    filename = Path(f"{name}.tflite")
    filename.write_bytes(tflite_model)

    size_kb = filename.stat().st_size / 1024
    print(f"{name}: {size_kb:.2f} KB")
```

```
convert_and_save_tflite(models["raw"][1], "encoder_raw")
convert_and_save_tflite(clf_raw, "clf_raw")
convert_and_save_tflite(models["std"][1], "encoder_std")
convert_and_save_tflite(clf_std, "clf_std")
convert_and_save_tflite(models["minmax"][1], "encoder_minmax")
convert_and_save_tflite(clf_minmax, "clf_minmax")
convert_and_save_tflite(meta_clf, "stacked_meta_clf")
```

INFO:tensorflow:Assets written to: /tmp/tmpkfhok8q6/assets

INFO:tensorflow:Assets written to: /tmp/tmpkfhok8q6/assets

2026-05-28 20:31:43.618507: W

tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.

2026-05-28 20:31:43.618572: W

tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.

2026-05-28 20:31:43.618965: I tensorflow/cc/saved_model/reader.cc:83]

Reading SavedModel from: /tmp/tmpkfhok8q6

2026-05-28 20:31:43.619384: I tensorflow/cc/saved_model/reader.cc:51]

Reading meta graph with tags { serve }

2026-05-28 20:31:43.619394: I tensorflow/cc/saved_model/reader.cc:146]

Reading SavedModel debug info (if present) from: /tmp/tmpkfhok8q6

2026-05-28 20:31:43.621068: I

tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:382] MLIR V1
optimization pass is not enabled

2026-05-28 20:31:43.621455: I tensorflow/cc/saved_model/loader.cc:233]

Restoring SavedModel bundle.

2026-05-28 20:31:43.637349: I tensorflow/cc/saved_model/loader.cc:217]

Running initialization op on SavedModel bundle at path:

/tmp/tmpkfhok8q6

2026-05-28 20:31:43.642518: I tensorflow/cc/saved_model/loader.cc:316]

SavedModel load for tags { serve }; Status: success: OK. Took 23554
microseconds.

2026-05-28 20:31:43.676503: I

tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269]

disabling MLIR crash reproducer, set env var

`MLIR\_CRASH\_REPRODUCER\_DIRECTORY` to enable.

encoder_raw: 159.88 KB

INFO:tensorflow:Assets written to: /tmp/tmpous24zps/assets

INFO:tensorflow:Assets written to: /tmp/tmpous24zps/assets

clf_raw: 5.41 KB

2026-05-28 20:31:44.102891: W

tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]

Ignored output_format.

2026-05-28 20:31:44.102947: W

```
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:31:44.103100: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpous24zps
2026-05-28 20:31:44.103808: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:44.103824: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpous24zps
2026-05-28 20:31:44.106053: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:44.130158: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpous24zps
2026-05-28 20:31:44.137043: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 33942
microseconds.
```

INFO:tensorflow:Assets written to: /tmp/tmp71m2fysu/assets

INFO:tensorflow:Assets written to: /tmp/tmp71m2fysu/assets

encoder_std: 159.88 KB

```
2026-05-28 20:31:44.559373: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:31:44.559492: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:31:44.559800: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmp71m2fysu
2026-05-28 20:31:44.560419: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:44.560441: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmp71m2fysu
2026-05-28 20:31:44.561604: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:44.575191: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmp71m2fysu
2026-05-28 20:31:44.581719: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 21921
microseconds.
```

INFO:tensorflow:Assets written to: /tmp/tmpsi95mbvt/assets

INFO:tensorflow:Assets written to: /tmp/tmpsi95mbvt/assets

clf_std: 5.41 KB

2026-05-28 20:31:44.945059: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:31:44.945117: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:31:44.945259: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpsi95mbvt
2026-05-28 20:31:44.945704: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:44.945714: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpsi95mbvt
2026-05-28 20:31:44.947106: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:44.974927: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpsi95mbvt
2026-05-28 20:31:44.983723: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 38463
microseconds.

INFO:tensorflow:Assets written to: /tmp/tmpdl4h860y/assets

INFO:tensorflow:Assets written to: /tmp/tmpdl4h860y/assets

encoder_minmax: 159.88 KB

2026-05-28 20:31:45.306850: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:31:45.306934: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:31:45.307084: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpdl4h860y
2026-05-28 20:31:45.307683: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:45.307700: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpdl4h860y
2026-05-28 20:31:45.308939: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:45.320492: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpdl4h860y
2026-05-28 20:31:45.325498: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 18412
microseconds.

INFO:tensorflow:Assets written to: /tmp/tmpi20rbq70/assets

INFO:tensorflow:Assets written to: /tmp/tmpi20rbq70/assets

clf_minmax: 5.41 KB

```
2026-05-28 20:31:45.749921: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:31:45.749974: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:31:45.750114: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpi20rbq70
2026-05-28 20:31:45.750916: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:45.750931: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpi20rbq70
2026-05-28 20:31:45.752773: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:45.778189: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpi20rbq70
2026-05-28 20:31:45.786413: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 36298
microseconds.
```

INFO:tensorflow:Assets written to: /tmp/tmpa7pcpgcc/assets

```
INFO:tensorflow:Assets written to: /tmp/tmpa7pcpgcc/assets
2026-05-28 20:31:46.259574: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:31:46.259683: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
```

stacked_meta_clf: 6.50 KB

```
2026-05-28 20:31:46.259973: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpa7pcpgcc
2026-05-28 20:31:46.262436: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:31:46.262481: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpa7pcpgcc
2026-05-28 20:31:46.268404: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:31:46.299846: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpa7pcpgcc
2026-05-28 20:31:46.306855: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 46884
microseconds.
```

12. Create flattened encoder-classifier pipelines

For deployment, each branch can be represented as a single model that maps the 600-dimensional input window directly to a 12-class softmax output.

Each flattened branch has this structure:

600 input neurons -> Dense(64, ReLU) -> Dense(32, linear) -> Dense(20, ReLU) -> Dense(12, Softmax)

```
def combine_encoder_and_classifier(encoder, classifier, name):
    combined = Sequential(name=name)
    combined.add(Input(shape=(encoder.input_shape[1],),
name="window_input"))

    for layer in encoder.layers[1:]:
        combined.add(layer)

    for layer in classifier.layers:
        combined.add(layer)

    combined.compile(optimizer=Adam(learning_rate=0.001),
loss="categorical_crossentropy", metrics=["accuracy"])
    return combined

flattened_raw = combine_encoder_and_classifier(models["raw"][1],
clf_raw, "encoder_clf_raw_flat")
flattened_std = combine_encoder_and_classifier(models["std"][1],
clf_std, "encoder_clf_std_flat")
flattened_minmax = combine_encoder_and_classifier(models["minmax"][1],
clf_minmax, "encoder_clf_minmax_flat")
```

```
flattened_raw.summary()
```

Model: "encoder_clf_raw_flat"

Layer (type)	Output Shape	Param #
encoder_dense_64 (Dense)	(None, 64)	38464
latent_32 (Dense)	(None, 32)	2080
classifier_dense_20 (Dense)	(None, 20)	660
activity_softmax (Dense)	(None, 12)	252
Total params: 41456 (161.94 KB)		
Trainable params: 41456 (161.94 KB)		

Non-trainable params: 0 (0.00 Byte)

13. Pruning, QAT, and int8 TFLite export

This section applies a deployment-oriented compression pipeline:

1. Prune the model using a polynomial sparsity schedule.
2. Strip the pruning wrappers to create a regular Keras model with sparse weights.
3. Apply QAT to simulate quantization effects during fine-tuning.
4. Reapply pruning masks during QAT so previously pruned weights remain zero.
5. Convert the QAT model to a fully int8 TFLite model using representative data.
6. Evaluate the exported TFLite model.

```
from tensorflow_model_optimization.sparsity.keras import (
    prune_low_magnitude,
    PolynomialDecay,
    UpdatePruningStep,
    strip_pruning,
)
from tensorflow_model_optimization.quantization.keras import
quantize_model, quantize_scope
from
tensorflow_model_optimization.python.core.quantization.keras.quantize_
wrapper import QuantizeWrapper

PRUNE_EPOCHS = 5
QAT_EPOCHS = 5
PRUNE_END_STEP = 500
REPRESENTATIVE_SAMPLES = 200

def train_and_prune(model, X_train, y_train):
    pruning_schedule = PolynomialDecay(
        initial_sparsity=0.10,
        final_sparsity=0.80,
        begin_step=0,
        end_step=PRUNE_END_STEP,
    )

    pruned_model = prune_low_magnitude(model,
    pruning_schedule=pruning_schedule)
    pruned_model.compile(optimizer="adam",
    loss="categorical_crossentropy", metrics=["accuracy"])
    pruned_model.fit(
        X_train,
        y_train,
        epochs=PRUNE_EPOCHS,
        batch_size=BATCH_SIZE,
```

```

        callbacks=[UpdatePruningStep()],
        verbose=1,
    )
    return pruned_model

def extract_dense_weight_masks(model):
    masks = []
    for layer in model.layers:
        if isinstance(layer, Dense):
            weights = layer.get_weights()
            if weights:
                layer_masks = [(w != 0).astype(np.float32) for w in
weights]
                masks.append((layer.name, [w.copy() for w in weights],
layer_masks))
    return masks

class MaskEnforcerCallback(tf.keras.callbacks.Callback):
    def __init__(self, masks):
        super().__init__()
        self.masks = masks

    def on_train_batch_end(self, batch, logs=None):
        self.apply_masks()

    def on_epoch_end(self, epoch, logs=None):
        self.apply_masks()

    def apply_masks(self):
        for layer in self.model.layers:
            if not isinstance(layer, QuantizeWrapper):
                continue

            weights = layer.get_weights()
            if len(weights) < 2:
                continue

            for original_name, original_weights, masks in self.masks:
                if len(original_weights) < 2:
                    continue
                if weights[0].shape == original_weights[0].shape and
weights[1].shape == original_weights[1].shape:
                    updated_weights = [weights[0] * masks[0],
weights[1] * masks[1]] + weights[2:]
                    layer.set_weights(updated_weights)
                    break

```



```

def make_representative_data_gen(X_subset,
max_samples=REPRESENTATIVE_SAMPLES):
    count = min(len(X_subset), max_samples)
    indices = np.linspace(0, len(X_subset) - 1, count, dtype=int)

    def generator():
        for i in indices:
            yield X_subset[i:i + 1].astype(np.float32)]

    return generator

def compute_model_sparsity(model):
    total_weights = 0
    total_zeros = 0
    print("\nLayer-wise sparsity:")

    for layer in model.layers:
        weights = layer.get_weights()
        if not weights:
            continue

        kernel = weights[0]
        if kernel.ndim < 2:
            continue

        zeros = np.sum(kernel == 0)
        size = np.prod(kernel.shape)
        total_weights += size
        total_zeros += zeros
        print(f"{layer.name:30s}: {100.0 * zeros / size:6.2f}% sparse
({zeros}/{size})")

    total_sparsity = 100.0 * total_zeros / total_weights if
total_weights else 0.0
    print(f"Overall sparsity: {total_sparsity:.2f}%
({total_zeros}/{total_weights})")
    return total_sparsity

def evaluate_tflite_model(tflite_model_path, X_test, y_test):
    interpreter =
tf.lite.Interpreter(model_path=str(tflite_model_path))
    interpreter.allocate_tensors()

    input_details = interpreter.get_input_details()
    output_details = interpreter.get_output_details()
    input_index = input_details[0]["index"]
    output_index = output_details[0]["index"]
    input_scale, input_zero_point = input_details[0]["quantization"]

```

```

y_pred = []
for i in range(len(X_test)):
    input_data = X_test[i:i + 1].astype(np.float32)
    if input_details[0]["dtype"] == np.int8:
        input_data = np.round(input_data / input_scale +
input_zero_point).astype(np.int8)

    interpreter.set_tensor(input_index, input_data)
    interpreter.invoke()
    output_data = interpreter.get_tensor(output_index)
    y_pred.append(int(np.argmax(output_data)))

accuracy = float(np.mean(np.asarray(y_pred) == y_test))
print("\nTFLite classification report:")
print(classification_report(y_test, y_pred, zero_division=0))
print("TFLite confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print(f"TFLite accuracy: {accuracy:.4f}")
return accuracy

def prune_qat_export(model, X_train, y_train, X_test, y_test, name):
    print(f"\nCompression pipeline for {name}")

    pruned_model = train_and_prune(model, X_train, y_train)
    stripped_model = strip_pruning(pruned_model)
    dense_masks = extract_dense_weight_masks(stripped_model)

    with quantize_scope():
        qat_model = quantize_model(stripped_model)

    qat_model.compile(optimizer="adam",
loss="categorical_crossentropy", metrics=["accuracy"])
    mask_callback = MaskEnforcerCallback(dense_masks)

    print("Training QAT model with pruning-mask enforcement")
    qat_model.fit(
        X_train,
        y_train,
        epochs=QAT_EPOCHS,
        batch_size=BATCH_SIZE,
        callbacks=[mask_callback],
        verbose=1,
    )
    mask_callback.apply_masks()

    compute_model_sparsity(qat_model)

    converter = tf.lite.TFLiteConverter.from_keras_model(qat_model)

```

```

    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.representative_dataset =
make_representative_data_gen(X_train)
    converter.target_spec.supported_ops =
[tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.int8
    converter.inference_output_type = tf.int8

    tflite_model = converter.convert()
    output_path = Path(f"{name}_pruned_qat_int8.tflite")
    output_path.write_bytes(tflite_model)

    print(f"Saved {output_path.name}: {output_path.stat().st_size /
1024:.2f} KB")
    evaluate_tflite_model(output_path, X_test, np.argmax(y_test,
axis=1))
    return output_path

```

14. Export compressed branch models and the meta-classifier

The following cells produce four int8 TFLite models:

- Raw encoder-classifier branch
- Standard-scaled encoder-classifier branch
- Min-max-scaled encoder-classifier branch
- Stacked meta-classifier

```

compressed_model_paths = []

compressed_model_paths.append(
    prune_qat_export(
        flattened_raw,
        X_train_raw,
        y_train_raw_cat,
        X_test_raw,
        y_test_raw_cat,
        "encoder_clf_raw",
    )
)

compressed_model_paths.append(
    prune_qat_export(
        flattened_std,
        X_train_std,
        y_train_std_cat,
        X_test_std,
        y_test_std_cat,
        "encoder_clf_std",
    )
)

```

```

    )
)
compressed_model_paths.append(
    prune_qat_export(
        flattened_minmax,
        X_train_minmax,
        y_train_minmax_cat,
        X_test_minmax,
        y_test_minmax_cat,
        "encoder_clf_minmax",
    )
)
compressed_model_paths.append(
    prune_qat_export(
        meta_clf,
        X_train_ensemble,
        y_train_ensemble_cat,
        X_test_ensemble,
        y_test_ensemble_cat,
        "stacked_meta_clf",
    )
)

```

Compression pipeline for encoder_clf_raw

Epoch 1/5

84/84 [=====] - 3s 8ms/step - loss: 0.4401 - accuracy: 0.9338

Epoch 2/5

84/84 [=====] - 0s 3ms/step - loss: 0.9358 - accuracy: 0.8273

Epoch 3/5

84/84 [=====] - 0s 3ms/step - loss: 1.9790 - accuracy: 0.6446

Epoch 4/5

84/84 [=====] - 0s 5ms/step - loss: 3.1414 - accuracy: 0.4890

Epoch 5/5

84/84 [=====] - 1s 8ms/step - loss: 1.6727 - accuracy: 0.3630

Training QAT model with pruning-mask enforcement

Epoch 1/5

6/84 [=>.....] - ETA: 0s - loss: 2.1041 - accuracy: 0.2168 WARNING:tensorflow:Callback method `on\_train\_batch\_end` is slow compared to the batch time (batch time: 0.0047s vs `on\_train\_batch\_end` time: 0.0071s). Check your callbacks.

WARNING:tensorflow:Callback method `on\_train\_batch\_end` is slow compared to the batch time (batch time: 0.0047s vs `on\_train\_batch\_end` time: 0.0071s). Check your callbacks.

84/84 [=====] - 2s 18ms/step - loss: 1.8985 - accuracy: 0.2689

Epoch 2/5

84/84 [=====] - 1s 12ms/step - loss: 1.4553 - accuracy: 0.4052

Epoch 3/5

84/84 [=====] - 1s 11ms/step - loss: 1.2159 - accuracy: 0.5627

Epoch 4/5

84/84 [=====] - 1s 12ms/step - loss: 1.0839 - accuracy: 0.6208

Epoch 5/5

84/84 [=====] - 1s 11ms/step - loss: 0.9988 - accuracy: 0.6742

Layer-wise sparsity:

quant_encoder_dense_64 : 79.44% sparse (30505/38400)

quant_latent_32 : 79.44% sparse (1627/2048)

quant_classifier_dense_20 : 79.38% sparse (508/640)

quant_activity_softmax : 79.58% sparse (191/240)

Overall sparsity: 79.44% (32831/41328)

INFO:tensorflow:Assets written to: /tmp/tmpmmapxu13/assets

INFO:tensorflow:Assets written to: /tmp/tmpmmapxu13/assets

/home/metal/ai/projects/tinyml-arduino/lib/python3.11/site-packages/tensorflow/lite/python/convert.py:947: UserWarning: Statistics for quantized inputs were expected, but not specified; continuing anyway.

warnings.warn(

2026-05-28 20:32:01.693237: W

tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378] Ignored output_format.

2026-05-28 20:32:01.693287: W

tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381] Ignored drop_control_dependency.

2026-05-28 20:32:01.693460: I tensorflow/cc/saved_model/reader.cc:83]

Reading SavedModel from: /tmp/tmpmmapxu13

2026-05-28 20:32:01.696428: I tensorflow/cc/saved_model/reader.cc:51]

Reading meta graph with tags { serve }

2026-05-28 20:32:01.696454: I tensorflow/cc/saved_model/reader.cc:146]

Reading SavedModel debug info (if present) from: /tmp/tmpmmapxu13

2026-05-28 20:32:01.704275: I tensorflow/cc/saved_model/loader.cc:233]

Restoring SavedModel bundle.

2026-05-28 20:32:01.771124: I tensorflow/cc/saved_model/loader.cc:217]

Running initialization op on SavedModel bundle at path:

/tmp/tmpmmapxu13

2026-05-28 20:32:01.788361: I tensorflow/cc/saved_model/loader.cc:316]

SavedModel load for tags { serve }; Status: success: OK. Took 94901 microseconds.

fully_quantize: 0, inference_type: 6, input_inference_type: INT8, output_inference_type: INT8

INFO: Created TensorFlow Lite XNNPACK delegate for CPU.

Saved encoder_clf_raw_pruned_qat_int8.tflite: 44.55 KB

TFLite classification report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	823
1	1.00	1.00	1.00	823
2	1.00	1.00	1.00	823
3	0.87	1.00	0.93	823
4	0.44	0.90	0.59	823
5	1.00	0.04	0.07	562
6	0.73	1.00	0.84	531
7	0.00	0.00	0.00	593
8	0.29	0.43	0.34	823
9	0.98	1.00	0.99	823
10	0.96	0.98	0.97	823
11	0.57	0.80	0.67	209
accuracy			0.70	8479
macro avg	0.65	0.68	0.62	8479
weighted avg	0.66	0.70	0.64	8479

TFLite confusion matrix:

```
[[ 0  0  0  0  0  0  0  0  0 823  0  0  0]
 [ 0 823  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  0 823  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0 819  1  0  0  0  0  0  0  0  3]
 [ 0  1  0  12 737  0  0  0  28  2  0  0 43]
 [ 0  0  0  73  0 20 197 237  0  0 35  0]
 [ 0  0  0  0  0  0 531  0  0  0  0  0]
 [ 0  0  0  0 479  0  0  0 38  0  0 76]
 [ 0  3  0  1 460  0  0  0 356  0  0  3]
 [ 0  0  0  0  0  0  0  0  0 822  1  0]
 [ 0  0  0 16  0  0  0  0  0  0 807  0]
 [ 0  0  0 16 14  0  0  0  0 11  1 167]]
```

TFLite accuracy: 0.6964

Compression pipeline for encoder_clf_std

Epoch 1/5

84/84 [=====] - 2s 6ms/step - loss: 0.0108 - accuracy: 0.9960

Epoch 2/5

84/84 [=====] - 0s 5ms/step - loss: 0.0075 - accuracy: 0.9979

```

Epoch 3/5
84/84 [=====] - 0s 2ms/step - loss: 0.1314 -
accuracy: 0.9734
Epoch 4/5
84/84 [=====] - 0s 2ms/step - loss: 0.4540 -
accuracy: 0.8945
Epoch 5/5
84/84 [=====] - 0s 4ms/step - loss: 0.3974 -
accuracy: 0.8734
Training QAT model with pruning-mask enforcement
Epoch 1/5
 5/84 [>.....] - ETA: 0s - loss: 0.7477 -
accuracy: 0.7312 WARNING:tensorflow:Callback method
`on_train_batch_end` is slow compared to the batch time (batch time:
0.0048s vs `on_train_batch_end` time: 0.0073s). Check your callbacks.

WARNING:tensorflow:Callback method `on_train_batch_end` is slow
compared to the batch time (batch time: 0.0048s vs
`on_train_batch_end` time: 0.0073s). Check your callbacks.

84/84 [=====] - 5s 13ms/step - loss: 0.5046 -
accuracy: 0.8630
Epoch 2/5
84/84 [=====] - 1s 12ms/step - loss: 0.3567 -
accuracy: 0.9732
Epoch 3/5
84/84 [=====] - 1s 10ms/step - loss: 0.2923 -
accuracy: 0.9886
Epoch 4/5
84/84 [=====] - 1s 13ms/step - loss: 0.2432 -
accuracy: 0.9958
Epoch 5/5
84/84 [=====] - 1s 11ms/step - loss: 0.2042 -
accuracy: 0.9992

Layer-wise sparsity:
quant_encoder_dense_64      : 79.44% sparse (30505/38400)
quant_latent_32             : 79.44% sparse (1627/2048)
quant_classifier_dense_20   : 79.38% sparse (508/640)
quant_activity_softmax      : 79.58% sparse (191/240)
Overall sparsity: 79.44% (32831/41328)
INFO:tensorflow:Assets written to: /tmp/tmp86_0ue6r/assets

INFO:tensorflow:Assets written to: /tmp/tmp86_0ue6r/assets
/home/metal/ai/projects/tinyml-arduino/lib/python3.11/site-packages/
tensorflow/lite/python/convert.py:947: UserWarning: Statistics for
quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(
2026-05-28 20:32:16.328499: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]

```

```

Ignored output_format.
2026-05-28 20:32:16.328548: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:32:16.328697: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmp86_0ue6r
2026-05-28 20:32:16.330628: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:32:16.330649: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmp86_0ue6r
2026-05-28 20:32:16.336155: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:32:16.398187: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmp86_0ue6r
2026-05-28 20:32:16.417173: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 88475
microseconds.
fully_quantize: 0, inference_type: 6, input_inference_type: INT8,
output_inference_type: INT8

```

Saved encoder_clf_std_pruned_qat_int8.tflite: 44.55 KB

TFLite classification report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	823
1	1.00	1.00	1.00	823
2	1.00	1.00	1.00	823
3	1.00	1.00	1.00	823
4	0.99	0.83	0.90	823
5	1.00	1.00	1.00	562
6	1.00	1.00	1.00	531
7	0.96	1.00	0.98	593
8	0.96	1.00	0.98	823
9	0.90	1.00	0.95	823
10	1.00	1.00	1.00	823
11	0.98	0.92	0.95	209
accuracy			0.98	8479
macro avg	0.98	0.98	0.98	8479
weighted avg	0.98	0.98	0.98	8479

TFLite confusion matrix:

```

[[823  0  0  0  0  0  0  0  0  0  0  0]
 [  0 823  0  0  0  0  0  0  0  0  0  0]
 [  0  0 823  0  0  0  0  0  0  0  0  0]
 [  0  0  0 823  0  0  0  0  0  0  0  0]
 [  0  0  0  0 682  0  0 22 34 81  0  4]
 [  0  0  0  0  0 562  0  0  0  0  0  0]

```



```
[ 0 0 0 0 0 0 531 0 0 0 0 0]
[ 0 0 0 0 0 0 0 593 0 0 0 0]
[ 0 0 0 0 1 0 0 0 819 3 0 0]
[ 0 0 0 0 0 1 0 0 0 822 0 0]
[ 0 0 0 0 0 0 0 0 0 0 823 0]
[ 0 0 0 0 6 0 0 0 0 10 0 193]]
```

TFLite accuracy: 0.9809

Compression pipeline for encoder_clf_minmax

Epoch 1/5

84/84 [=====] - 2s 3ms/step - loss: 0.1569 -
accuracy: 0.9558

Epoch 2/5

84/84 [=====] - 1s 6ms/step - loss: 0.2016 -
accuracy: 0.9419

Epoch 3/5

84/84 [=====] - 1s 6ms/step - loss: 0.3880 -
accuracy: 0.8870

Epoch 4/5

84/84 [=====] - 1s 8ms/step - loss: 0.8926 -
accuracy: 0.6941

Epoch 5/5

84/84 [=====] - 1s 7ms/step - loss: 1.1842 -
accuracy: 0.6112

Training QAT model with pruning-mask enforcement

Epoch 1/5

6/84 [=>.....] - ETA: 0s - loss: 1.4109 -
accuracy: 0.4857 WARNING:tensorflow:Callback method
`on\_train\_batch\_end` is slow compared to the batch time (batch time:
0.0036s vs `on\_train\_batch\_end` time: 0.0080s). Check your callbacks.

WARNING:tensorflow:Callback method `on\_train\_batch\_end` is slow
compared to the batch time (batch time: 0.0036s vs
`on\_train\_batch\_end` time: 0.0080s). Check your callbacks.

84/84 [=====] - 2s 15ms/step - loss: 1.0653 -
accuracy: 0.6088

Epoch 2/5

84/84 [=====] - 1s 17ms/step - loss: 0.8282 -
accuracy: 0.6959

Epoch 3/5

84/84 [=====] - 1s 16ms/step - loss: 0.7174 -
accuracy: 0.8067

Epoch 4/5

84/84 [=====] - 1s 14ms/step - loss: 0.6347 -
accuracy: 0.8804

Epoch 5/5

84/84 [=====] - 1s 12ms/step - loss: 0.5695 -
accuracy: 0.8968

```

Layer-wise sparsity:
quant_encoder_dense_64      : 79.44% sparse (30505/38400)
quant_latent_32             : 79.44% sparse (1627/2048)
quant_classifier_dense_20   : 79.38% sparse (508/640)
quant_activity_softmax      : 79.58% sparse (191/240)
Overall sparsity: 79.44% (32831/41328)
INFO:tensorflow:Assets written to: /tmp/tmp6nenvok/assets

INFO:tensorflow:Assets written to: /tmp/tmp6nenvok/assets
/home/metal/ai/projects/tinyml-arduino/lib/python3.11/site-packages/
tensorflow/lite/python/convert.py:947: UserWarning: Statistics for
quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(
2026-05-28 20:32:29.940292: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:32:29.940394: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:32:29.940679: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmp6nenvok
2026-05-28 20:32:29.943572: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:32:29.943601: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmp6nenvok
2026-05-28 20:32:29.951403: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:32:30.010090: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmp6nenvok
2026-05-28 20:32:30.027108: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 86431
microseconds.
fully_quantize: 0, inference_type: 6, input_inference_type: INT8,
output_inference_type: INT8

Saved encoder_clf_minmax_pruned_qat_int8.tflite: 44.61 KB

```

TFLite classification report:

	precision	recall	f1-score	support
0	0.97	1.00	0.99	823
1	1.00	1.00	1.00	823
2	1.00	1.00	1.00	823
3	0.90	0.99	0.94	823
4	0.44	0.39	0.41	823
5	0.39	0.37	0.38	562
6	1.00	1.00	1.00	531
7	0.88	0.99	0.93	593
8	0.99	0.93	0.96	823

9	0.99	1.00	1.00	823
10	1.00	1.00	1.00	823
11	0.78	0.78	0.78	209
accuracy			0.88	8479
macro avg	0.86	0.87	0.87	8479
weighted avg	0.88	0.88	0.88	8479

TFLite confusion matrix:

```
[[823  0  0  0  0  0  0  0  0  0  0  0]
 [  0 823  0  0  0  0  0  0  0  0  0  0]
 [  0  0 823  0  0  0  0  0  0  0  0  0]
 [  0  0  0 812  4  0  0  1  6  0  0  0]
 [ 23  0  0  40 318 329  1 66  0  0  0 46]
 [  0  0  0  0 356 206  0  0  0  0  0  0]
 [  0  0  0  0  0  0 531  0  0  0  0  0]
 [  0  0  0  3  0  0  0 588  2  0  0  0]
 [  0  0  0 50  0  0  0  10 763  0  0  0]
 [  0  0  0  0  0  0  0  0  0 823  0  0]
 [  0  0  0  0  0  0  0  0  1  0 822  0]
 [  0  0  0  0 40  0  0  0  0  5  0 164]]
```

TFLite accuracy: 0.8841

Compression pipeline for stacked_meta_clf

Epoch 1/5

84/84 [=====] - 1s 3ms/step - loss: 6.3123e-05 - accuracy: 1.0000

Epoch 2/5

84/84 [=====] - 0s 4ms/step - loss: 0.0018 - accuracy: 1.0000

Epoch 3/5

84/84 [=====] - 0s 4ms/step - loss: 0.0253 - accuracy: 1.0000

Epoch 4/5

84/84 [=====] - 0s 4ms/step - loss: 0.1690 - accuracy: 0.9397

Epoch 5/5

84/84 [=====] - 0s 5ms/step - loss: 0.1743 - accuracy: 0.9757

Training QAT model with pruning-mask enforcement

Epoch 1/5

1/84 [.....] - ETA: 48s - loss: 0.2084 - accuracy: 1.0000
WARNING:tensorflow:Callback method `on\_train\_batch\_end` is slow compared to the batch time (batch time: 0.0018s vs `on\_train\_batch\_end` time: 0.0033s). Check your callbacks.

WARNING:tensorflow:Callback method `on\_train\_batch\_end` is slow compared to the batch time (batch time: 0.0018s vs `on\_train\_batch\_end` time: 0.0033s). Check your callbacks.

```

84/84 [=====] - 1s 6ms/step - loss: 0.1559 -
accuracy: 1.0000
Epoch 2/5
84/84 [=====] - 0s 6ms/step - loss: 0.1021 -
accuracy: 1.0000
Epoch 3/5
84/84 [=====] - 0s 5ms/step - loss: 0.0740 -
accuracy: 1.0000
Epoch 4/5
84/84 [=====] - 1s 6ms/step - loss: 0.0553 -
accuracy: 1.0000
Epoch 5/5
84/84 [=====] - 1s 7ms/step - loss: 0.0420 -
accuracy: 1.0000

```

Layer-wise sparsity:

quant_meta_dense_24 : 79.40% sparse (686/864)

quant_meta_softmax : 79.51% sparse (229/288)

Overall sparsity: 79.43% (915/1152)

INFO:tensorflow:Assets written to: /tmp/tmpdpl8ojvs/assets

INFO:tensorflow:Assets written to: /tmp/tmpdpl8ojvs/assets

Saved stacked_meta_clf_pruned_qat_int8.tflite: 3.63 KB

TFLite classification report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	823
1	1.00	1.00	1.00	823
2	1.00	1.00	1.00	823
3	1.00	1.00	1.00	823
4	0.99	0.91	0.95	823
5	1.00	1.00	1.00	562
6	1.00	1.00	1.00	531
7	1.00	1.00	1.00	593
8	0.93	1.00	0.97	823
9	1.00	1.00	1.00	823
10	1.00	1.00	1.00	823
11	1.00	0.96	0.98	209
accuracy			0.99	8479
macro avg	0.99	0.99	0.99	8479
weighted avg	0.99	0.99	0.99	8479

TFLite confusion matrix:

```

[[823  0  0  0  0  0  0  0  0  0  0  0]
 [  0 823  0  0  0  0  0  0  0  0  0  0]
 [  0  0 823  0  0  0  0  0  0  0  0  0]
 [  0  0  0 823  0  0  0  0  0  0  0  0]

```

```
[ 14  0  0  0 748  0  1  1 59  0  0  0]
[  0  0  0  0  0 562  0  0  0  0  0  0]
[  0  0  0  0  0  0 531  0  0  0  0  0]
[  0  0  0  0  0  0  0 593  0  0  0  0]
[  0  0  0  0  0  0  0  0 823  0  0  0]
[  0  0  0  0  0  0  0  0  0 823  0  0]
[  0  0  0  0  0  0  0  0  0  0 823  0]
[  0  0  0  0  8  0  0  0  0  0  0 201]]
```

TFLite accuracy: 0.9902

```
/home/metal/ai/projects/tinymml-arduino/lib/python3.11/site-packages/
tensorflow/lite/python/convert.py:947: UserWarning: Statistics for
quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(
2026-05-28 20:32:37.044843: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378]
Ignored output_format.
2026-05-28 20:32:37.044893: W
tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381]
Ignored drop_control_dependency.
2026-05-28 20:32:37.045040: I tensorflow/cc/saved_model/reader.cc:83]
Reading SavedModel from: /tmp/tmpdpl8ojvs
2026-05-28 20:32:37.046715: I tensorflow/cc/saved_model/reader.cc:51]
Reading meta graph with tags { serve }
2026-05-28 20:32:37.046737: I tensorflow/cc/saved_model/reader.cc:146]
Reading SavedModel debug info (if present) from: /tmp/tmpdpl8ojvs
2026-05-28 20:32:37.052127: I tensorflow/cc/saved_model/loader.cc:233]
Restoring SavedModel bundle.
2026-05-28 20:32:37.091861: I tensorflow/cc/saved_model/loader.cc:217]
Running initialization op on SavedModel bundle at path:
/tmp/tmpdpl8ojvs
2026-05-28 20:32:37.102565: I tensorflow/cc/saved_model/loader.cc:316]
SavedModel load for tags { serve }; Status: success: OK. Took 57524
microseconds.
fully_quantize: 0, inference_type: 6, input_inference_type: INT8,
output_inference_type: INT8
```

15. Report compressed model sizes

```
print("Compressed int8 TFLite model sizes")
print(f"{'Model':45s} {'Size (KB)':>10s}")
print("-" * 58)

for path in compressed_model_paths:
    path = Path(path)
    if path.exists():
        print(f"{path.name:45s} {path.stat().st_size / 1024:10.2f}")
    else:
        print(f"{path.name:45s} {'missing':>10s}")
```

Compressed int8 TFLite model sizes	
Model	Size (KB)

encoder_clf_raw_pruned_qat_int8.tflite	44.55
encoder_clf_std_pruned_qat_int8.tflite	44.55
encoder_clf_minmax_pruned_qat_int8.tflite	44.61
stacked_meta_clf_pruned_qat_int8.tflite	3.63

16. Convert TFLite files to C arrays for Arduino deployment

The Arduino sketch can include model files as C arrays. This cell uses `xxd`, which is available by default on macOS and Linux.

```
import subprocess

for path in compressed_model_paths:
    path = Path(path)
    output_cc = path.with_suffix(".cc")
    if path.exists():
        with output_cc.open("w") as f:
            subprocess.run(["xxd", "-i", str(path)], stdout=f,
check=True)
            print(f"Created {output_cc.name}")
    else:
        print(f"Skipped {path.name}; file not found")

Created encoder_clf_raw_pruned_qat_int8.cc
Created encoder_clf_std_pruned_qat_int8.cc
Created encoder_clf_minmax_pruned_qat_int8.cc
Created stacked_meta_clf_pruned_qat_int8.cc
```

17. Scaling parameters for Arduino implementation

The Arduino sketch needs the same scaling parameters used during training. Use these printed values when implementing standard scaling and min-max scaling on the board.

```
print("Standard-scaling parameters")
for feature, mean, std in zip(sensor_features, std_scaler.mean_,
np.sqrt(std_scaler.var_)):
    print(f"{feature:8s}: mean = {mean: .6f}, std = {std: .6f}")

print("\nMin-max-scaling parameters")
for feature, min_val, max_val in zip(sensor_features,
minmax_scaler.data_min_, minmax_scaler.data_max_):
    print(f"{feature:8s}: min = {min_val: .6f}, max = {max_val: .6f}")

Standard-scaling parameters
Accel_X : mean =  3.268805, std =  4.253204
```

```
Accel_Y : mean = -9.153526, std = 5.397565
Accel_Z : mean = 0.700608, std = 6.433631
Gyro_X  : mean = -0.020571, std = 0.325429
Gyro_Y  : mean = -0.520382, std = 0.571098
Gyro_Z  : mean = -0.299817, std = 0.542104
```

Min-max-scaling parameters

```
Accel_X : min = -22.091000, max = 20.003000
Accel_Y : min = -19.570999, max = 20.909000
Accel_Z : min = -19.363001, max = 24.599001
Gyro_X  : min = -0.977740, max = 1.463800
Gyro_Y  : min = -1.611600, max = 1.557200
Gyro_Z  : min = -1.206300, max = 1.326100
```

18. Optional recorded-IMU validation

If you recorded a new IMU file named `imu_500_rows.csv`, this section runs the trained Keras models on that file. The expected CSV columns are:

`acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z`

This section is useful for checking whether real board data behaves similarly to the training data. If the CSV file is not present, the cell will skip this validation.

```
RECORDED_IMU_FILE = Path("imu_500_rows.csv")

if not RECORDED_IMU_FILE.exists():
    print("imu_500_rows.csv was not found. Skipping recorded-IMU
validation.")
else:
    df_new = pd.read_csv(RECORDED_IMU_FILE)
    df_new = df_new.rename(columns={
        "acc_x": "Accel_X",
        "acc_y": "Accel_Y",
        "acc_z": "Accel_Z",
        "gyro_x": "Gyro_X",
        "gyro_y": "Gyro_Y",
        "gyro_z": "Gyro_Z",
    })

    missing_columns = [col for col in sensor_features if col not in
df_new.columns]
    if missing_columns:
        raise ValueError(f"Missing required columns in
imu_500_rows.csv: {missing_columns}")

    X_new_raw_features =
df_new[sensor_features].to_numpy(dtype=np.float32)
    X_new_std_features =
```

```

std_scaler.transform(X_new_raw_features).astype(np.float32)
X_new_minmax_features =
minmax_scaler.transform(X_new_raw_features).astype(np.float32)

def create_windows_from_array(X, window_size=WINDOW_SIZE,
stride=STRIDE):
    return np.asarray(
        [X[start:start + window_size].reshape(-1) for start in
range(0, len(X) - window_size + 1, stride)],
        dtype=np.float32,
    )

X_raw_new = create_windows_from_array(X_new_raw_features)
X_std_new = create_windows_from_array(X_new_std_features)
X_minmax_new = create_windows_from_array(X_new_minmax_features)

if len(X_raw_new) == 0:
    raise ValueError("The recorded IMU file does not contain
enough rows to create one full window.")

logits_raw = flattened_raw.predict(X_raw_new, verbose=0)
logits_std = flattened_std.predict(X_std_new, verbose=0)
logits_minmax = flattened_minmax.predict(X_minmax_new, verbose=0)

pred_raw = np.argmax(logits_raw, axis=1)
pred_std = np.argmax(logits_std, axis=1)
pred_minmax = np.argmax(logits_minmax, axis=1)

X_meta_new = np.hstack([logits_raw, logits_std,
logits_minmax]).astype(np.float32)
meta_logits = meta_clf.predict(X_meta_new, verbose=0)
pred_meta = np.argmax(meta_logits, axis=1)

print("Recorded-IMU input windows:")
print("Raw:", X_raw_new.shape)
print("Standard scaled:", X_std_new.shape)
print("Min-max scaled:", X_minmax_new.shape)

print("\nFirst 10 predicted class IDs")
print("Raw branch: ", pred_raw[:10])
print("Standard-scaled branch: ", pred_std[:10])
print("Min-max-scaled branch: ", pred_minmax[:10])
print("Stacked meta-classifier:", pred_meta[:10])

print("\nFirst 10 stacked meta-classifier labels")
print([label_map[int(label)] for label in pred_meta[:10]])

imu_500_rows.csv was not found. Skipping recorded-IMU validation.

```